

RESEARCH ARTICLE

Model selection with bootstrap validation

Rafael Savvides¹ | Jarmo Mäkelä¹ | Kai Puolamäki²¹Department of Computer Science,
University of Helsinki, Helsinki, Finland²Department of Computer Science,
University of Helsinki and Institute for
Atmospheric and Earth System Research,
University of Helsinki, Helsinki, Finland

Correspondence

Rafael Savvides, Department of Computer
Science, University of Helsinki, Helsinki,
Finland.Email: rafael.savvides@helsinki.fi

Funding information

Academy of Finland; Helsingin Yliopisto;
Jane ja Aatos Erkon Säätiö; Future Makers
Program of the Technology Industries of
Finland Centennial Foundation

Abstract

Model selection is one of the most central tasks in supervised learning. Validation set methods are the standard way to accomplish this task: models are trained on training data, and the model with the smallest loss on the validation data is selected. However, it is generally not obvious how much validation data is required to make a reliable selection, which is essential when labeled data are scarce or expensive. We propose a bootstrap-based algorithm, *bootstrap validation* (BSV), that uses the bootstrap to adjust the validation set size and to find the best-performing model within a tolerance parameter specified by the user. We find that BSV works well in practice and can be used as a drop-in replacement for validation set methods or k -fold cross-validation. The main advantage of BSV is that less validation data is typically needed, so more data can be used to train the model, resulting in better approximations and efficient use of validation data.

KEYWORDS

bootstrap, model selection

1 | INTRODUCTION

In machine learning, the model selection task is to select a model from a set of candidate models (or a model class), such that the performance of the chosen model is maximized on new data. Model selection is a fundamental task for which many approaches exist in the literature; see [14] for discussion.

The textbook standard for model selection is the hold-out approach: split the data into training and validation sets, train the models on the training set, and then choose a model which performs the best on the validation set. Another standard is cross-validation (CV), which utilizes the data more efficiently but at the computational cost imposed by the need to train the model several times—one model trained for each CV fold.

A central question with validation methods is to estimate how much data is needed for training the models and

how much data should be set aside for validation purposes. On the one hand, we would like to make the training set as large as possible to train the best models, but on the other hand, have a large enough validation set to choose the best model reliably. However, it is non-trivial to find a statistical guarantee that ensures we are not making a random choice caused by a too small validation set. Indeed, determining the correct validation set size and the over-fitting caused by data reuse are significant issues in machine learning model evaluation [20].

The contribution of this paper is a method, called *bootstrap validation* (BSV) that selects a “good enough” model from k given models using as little data as possible. BSV is of general use and can be used as a drop-in replacement for the standard validation set methods.

The data efficiency provided by BSV is essential when data used to evaluate the model are scarce, such as when collecting new data is expensive or slow. As shown in

our experiments, BSV often uses a substantially smaller amount of data than conventional methods, such as the holdout approach. As a result, we can increase the training set size, which results in better models, or use the remaining validation set items later for other purposes, such as estimating generalization performance.

The main idea in BSV is to automatically adjust the validation set size until we reach a user-specified tolerance and statistical guarantee. The statistical guarantee, specified by a parameter α , gives an upper bound for the probability that the selected model is not “good enough.” The tolerance, specified by a parameter δ , defines that a model is “good enough” if its actual loss is within δ from the best out of the k models. BSV works with arbitrary loss functions and can be used in most supervised learning setups. The data efficiency of BSV is due to (i) selecting a “good enough” model instead of “the best” and (ii) being based on bootstrapping the validation set, which automatically takes into account correlations between model predictions.

The paper is organized as follows: Section 1.1 discusses potential applications of BSV. Section 1.2 reviews related work on validation set methods. Section 2 describes the problem of finding a model that is approximately the best, introduces the BSV algorithm for solving the problem, and shows its theoretical properties. Section 3 shows experimentally that the algorithm works as described and can be used as a drop-in replacement for validation set methods. Section 4 contains discussion and future work.

1.1 | Applications of BSV

The main application of BSV is model selection, but it can also be applied in various other settings. We next discuss some of the potential applications.

1.1.1 | Replacement for validation set methods

The most obvious application for BSV is to use it as a drop-in replacement for validation set methods. The advantage of BSV is that by specifying the required performance parameters, it automatically adjusts the required size of the validation set to be as small as possible and simultaneously sufficient to reach the required statistical guarantee. The potential drawback of BSV is that there is a non-zero probability that the algorithm needs more validation data than what is available. However, if this happens, BSV becomes equivalent to the traditional training-validation set split without statistical guarantees (the holdout approach). Another advantage of BSV, especially compared to k -fold CV, is that it is often

necessary to train the model only once, as opposed to k times in k -fold CV.

1.1.2 | Model hyper-parameter tuning

Related to the above application, BSV can also be used to tune hyper-parameters within a model family with the same advantages. BSV is especially useful for hyper-parameter tuning, as predictions from models with slightly different hyper-parameters are often highly correlated. Since the bootstrap uses the joint empirical distribution of the model predictions, it will take their correlations into account and count models with identical predictions effectively as one, resulting in efficient data use. An example of applying BSV for hyper-parameter tuning is given in Section 3.4.

1.1.3 | Concept drift detection

An interesting topic for future study is concept drift detection [13, 25]. In this scenario, we have trained a set of supervised learning (classification or regression) models and know the models that perform well on the original data. When the models are applied in real-life settings, it is essential to detect if the data distribution has changed in a way that affects the relative performance of the machine learning models. Even if there is a large volume of data, labeled data is often scarce and expensive. An example of such a scenario is sensor calibration, where the idea is to use regression models to “calibrate” the measurements of low-cost sensors [5]. In this application, the performance of a low-cost sensor is affected by environmental conditions; these interactions are captured and modeled by the regression models. Reference measurements (new labeled data points) for the low-cost sensors are usually expensive to produce, so it is vital to use the scarce labeled data items sparingly. We can utilize BSV in this setup by feeding the reference measurements to the algorithm. If BSV detects that some other model starts to perform better, we may conclude that concept drift has occurred and the model may have to be re-trained.

1.1.4 | Interactive data exploration

BSV is useful in all setups where it is advantageous to save validation data for further uses. One such generic scenario is interactive data analysis. A typical workflow in such a setup is a user (or algorithm) that sees a view in the data, based on which they proceed to the next step, obtaining another view, and so forth; see, for example, [15, 28, 29]. During the exploration, the user can observe various patterns in the data. It would be useful if there were a

holdout dataset that could be used to test whether an observed pattern is real or just a random artifact [32]. Data efficient validation methods, such as BSV can support such interactive data exploration by allowing the user to test more hypotheses with the holdout data than is possible with traditional methods.

1.1.5 | Privacy-preserving machine learning

In many applications, it is essential to validate models while simultaneously revealing as little information as possible from the underlying data distribution, such as when the data contain sensitive information. One way to approach the problem is to introduce carefully controlled noise to the validation data [9, 11] so that it is possible to select the best model yet minimize information leakage. BSV offers an alternative approach: instead of adding noise to the data, we can equivalently try to minimize the amount of data used for validation. Using less data for validation BSV can limit the information leakage compared to using the whole validation data.

1.2 | Related work

Model selection is an essential task in machine learning. There is a plethora of model selection methods that can be mainly differentiated into probabilistic methods, such as Akaike or Bayesian Information Criteria (AIC or BIC), or re-sampling methods, such as CV and bootstrapping—see [8] for a review.

Our algorithm uses the bootstrap to select a “good enough” model. Although the bootstrap has been used for model selection previously, the focus has been on choosing optimal features [2, 34], reducing the variance of bootstrap estimates [17], accuracy estimation with a dataset of a given size [16] or after model selection [10]. In contrast, we use a bootstrap confidence interval to obtain a p -value for a derived loss statistic inside an iterative significance testing procedure.

The iterative nature of our algorithm (increasing the validation set until a criterion is satisfied) is similar to early stopping. Early stopping is often seen in the context of neural networks [27], but has also been used to speed up hyper-parameter optimization with adaptive resource allocation [19]. Furthermore, there are similarities to the Hoeffding races algorithm [21, 22] that iterates over validation points and eliminates candidates whose lowest error estimate is worse than the highest of the best one. The clinical trial literature also contains various methods that try to minimize data use in order to terminate a drug experiment after enough evidence has been collected on the efficacy of the treatment [26].

The model selection literature includes other related topics, such as over-fitting prevention with the reusable holdout [9], controlling bias via limiting information usage [31], and various data partition practices [3, 7, 16]. The latter is a major point of interest for various CV techniques [1]. While the literature contains various approaches for model selection of classifiers, they tend to either focus on the case of multiple datasets [6] or make simplistic assumptions on the stopping criteria (boundary behavior) when selecting a winner between two classifiers [18].

The probability bounds given for the model output by our algorithm are similar to bounds in prior work, such as in Rashomon sets [33] (the set of almost-equally-accurate models for a given problem), and predictive multiplicity [23].

2 | METHODS

In this paper, we study the problem of finding a “good enough” supervised learning (classification or regression) model with minimal data. In this section, we formulate the problem, propose the BSV algorithm for solving it, and show the theoretical properties of the algorithm.

2.1 | Problem definition

We assume we can draw i.i.d. data points (x, y) at random from an unknown but fixed distribution, where x is a covariate (e.g., a real or categorical vector) and y is the discrete class variable (for classifiers), probability (for probabilistic classifiers), or a real number (for regression).

We assume that there are k distinct and fixed supervised learning models that have been trained beforehand with separate training data and that all try to estimate y given x , that is, $\hat{y}_j = f_j(x)$, where $j \in [k]$ and where we use the short-hand notation $[k] = \{1, \dots, k\}$. The functions f_j output discrete classes or class probabilities for classification tasks and real numbers for regression tasks. The (true) loss of a model j is defined by $\text{loss}_j = E [L(\hat{y}_j, y)]$, where L is a loss function, and the expectation is over sampling of a new data point (x, y) from the distribution.

We can choose the loss function L freely, but in this paper, we will consider (i) classifier models that output the predicted class $\hat{y} \in [K]$, where K is the number of classes (ii) probabilistic classifiers that output a simplex vector of class probabilities $\hat{y} \in \Delta^K$,¹ and (iii) regression models that output real numbers $\hat{y} \in \mathbb{R}$. The respective loss functions are (i) 0–1 loss $L_{01}(\hat{y}, y) = I(\hat{y} \neq y)$ for classification,

¹ Δ^K denotes the simplex $\Delta^K = \{p \in \mathbb{R}_{\geq 0}^K \mid \sum_{i=1}^K p_i = 1\}$.

where $y \in [K]$ and $I(z)$ is the indicator function that equals unity if z is true and zero otherwise, (ii) clipped log-loss $L_{\log}(\hat{p}_y, y) = \min(-\log \hat{p}_y, L_{\max})$ for probabilistic classification, where $\hat{p}_y$ is the estimated probability for class y and $L_{\max}$ is a positive real constant, and (iii) quadratic loss $L_2(\hat{y}, y) = (\hat{y} - y)^2$, where $y \in \mathbb{R}$, for regression. We assume throughout that the value of the loss is finite and has a well-defined mean and variance. This is why we use clipped log loss instead of raw log loss, which may, in practice, have infinite values. Notice that a well-defined mean is required for any validation set method to work; no validation method can work if the mean of the loss is ill-defined or cannot be estimated with a finite validation set.

Given a parameter δ and two models $i, j \in [k]$, we say that the models i and j are *approximately as good* if their (true) losses are within δ from each other, or $|\text{loss}_i - \text{loss}_j| \leq \delta$. We say that a model is *approximately the best* if it is approximately as good as the model with the lowest loss $\text{loss}_{\min} = \min_{i \in [k]} \text{loss}_i$. We now formally define the main problem addressed in this paper:

Problem 1. Given δ, α , and the definitions above, find a model $i_{\text{alg}} \in [k]$ that is approximately the best with a probability of at least $1 - \alpha$, or:

$$\Pr\left(|\text{loss}_{i_{\text{alg}}} - \text{loss}_{\min}| \leq \delta\right) \geq 1 - \alpha.$$

If our algorithm outputs a model that is not approximately the best, we say that the algorithm made an *error*. We denote the set of approximately the best models by

$$\mathcal{B} = \{i \in [k] \mid |\text{loss}_i - \text{loss}_{\min}| \leq \delta\}. \quad (1)$$

Our objective is to solve Problem 1 using as few data points as possible. We next describe the algorithms used to solve it in Section 2.2.

2.2 | BSV: Finding a model that is approximately the best

Our algorithm BSV (Algorithm 1) works as follows: The algorithm takes as an input the desired significance level α , the tolerance parameter δ , as defined in Problem 1, the initial number of data points T to sample, and the maximum number of data points $T_{\max}$ available for sampling. We let D_0 be T data points $(1, x_1, y_1), \dots, (T, x_T, y_T)$ sampled i.i.d. from the data distribution (line 1). Here the data points are described by triplets (j, x, y) , where j denotes the unique integer index of the data point.

We denote the empirical loss of model $i \in [k]$ on a dataset D by

Algorithm 1. Bootstrap validation (BSV)

input : α : significance level; δ : tolerance

parameter; T : initial number of data points; $T_{\max}$: maximum number of data points, where $2T \leq T_{\max}$.

output: A model i that is approximately the best with a probability of at least $1 - \alpha$.

```

1 Let  $D_0 \leftarrow \{(j, x_j, y_j)\}_{j=1}^T$  be a set of  $T$  newly
  sampled data points.;
2  $t \leftarrow 0$ ;
3 repeat
4    $t \leftarrow t + 1$ ;
5   /* Find the candidate model  $i_C$ . */
6    $i_C \leftarrow \arg \min_{i \in [k]} L_i(D_0)$ ;
7   Let  $D_1 \leftarrow \{(j, x_j, y_j)\}_{j=|D_0|+1}^{\min(T_{\max}, 2|D_0|)}$  be a set of at
    most  $|D_0|$  newly sampled data points.;
8   /* Compute a p-value. Adjust the
    number of bootstraps so we can
    detect p-values  $< 2^{-t}\alpha$ . */
9    $p_t \leftarrow \text{bootstrapPvalue}(i_C, D_1, \left\lceil \frac{10}{(2^{-t}\alpha)} \right\rceil, \delta)$ ;
10   $D_0 \leftarrow D_0 \cup D_1$ ;
11 until  $p_t \leq 2^{-t}\alpha$  or  $|D_0| = T_{\max}$ ;
12 if  $p_t > 2^{-t}\alpha$  then
13   Output a warning:  $T_{\max}$  was reached, and we
    are unable to give a statistical guarantee.;
14 return  $i_C$ 
```

$$L_i(D) = \frac{1}{|D|} \sum_{(j, x, y) \in D} L(f_i(x), y). \quad (2)$$

Furthermore, we define our *test statistic* for a dataset D and model $i \in [k]$ to be

$$s_i(D) = L_i(D) - \min_{i' \neq i} L_{i'}(D) = \max_{i' \neq i} (L_i(D) - L_{i'}(D)), \quad (3)$$

where $i' \neq i$ means $i' \in [k] \setminus \{i\}$, and where we have used the empirical loss defined by Equation (2).

We observe that the model $i \in [k]$ is approximately the best, according to the definition of Problem 1, if and only if, at the limit of an infinitely large D , the inequality $S = \lim_{|D| \rightarrow \infty} s_i(D) \leq \delta$ is satisfied. Notice that if the loss has a finite and well-defined mean, then S is always uniquely defined.

At each iteration $t \in \{1, 2, \dots\}$, the algorithm uses the data in D_0 to find a candidate model $i_C^t = \arg \min_{i \in [k]} L_i(D_0)$ (line 5) and samples $2^{t-1}T$ new data points to D_1 from the distribution (line 6). The data points in D_1 are used by the bootstrap procedure of Algorithm 2

Algorithm 2. The bootstrap p value algorithm

input : i_C : the candidate model; D : data set; M : number of bootstrap samples; δ : tolerance parameter; T_{min} : minimum sample size, by default $T_{min} = T$ from Alg. 1.

output: A p -value p for the null hypothesis that the model i_C is not approximately the best.

```

1 if  $|D| > T_{min}$  then
2    $\hat{S} \leftarrow s_{i_C}(D)$ ; /*  $s_{i_C}$  in Equation (3). */
3   for  $m = 1$  to  $M$  do
4     Create a bootstrapped dataset  $D_m^*$  of size  $|D|$ 
      by randomly sampling the data items from
       $D$  with replacement;
5      $S_m^* \leftarrow s_{i_C}(D_m^*)$ ; /* Bootstrap
      distribution of the test
      statistic. */
      /* Compute empirical  $p$ -value
      [24].  $I(\square)$  is 1 if  $\square$  is true
      else it is 0. */
6    $p \leftarrow (1 + \sum_{m=1}^M I(S_m^* \leq 2\hat{S} - \delta)) / (1 + M)$ ;
7 else
8   /* If  $D$  is too small, do not run
      the bootstrap. */
9    $p \leftarrow 1$ 
10 return  $p$ 

```

to find a p -value p_t for the null hypothesis that the candidate model i_C^t is not approximately the best (line 7). If $p_t \leq 2^{-t}\alpha$ the algorithm terminates and outputs i_C^t ; otherwise, the counter t is incremented by one, D_1 is appended to D_0 , and the iteration is repeated.

If the algorithm terminates because it reached T_{max} (the upper limit on the data points to be sampled), and if the bootstrap p -value does not satisfy the termination condition, then a warning is output that we cannot give a level- α statistical guarantee that the output model is approximately the best (line 11). We observe that the algorithm uses $\sim \min(T_{max}, 2^t \times T)$ data points, where t is the number of iterations run.

When T_{max} is reached, the algorithm has used $T_{max}/2$ validation points to select a model and $T_{max}/2$ to test if it is approximately the best. In this case, we can instead use all T_{max} points to select the best model, and BSV then reduces to the holdout approach.

If the data fed into the bootstrap procedure is too small (line 1 of Algorithm 2), then the bootstrap is not run and a p -value $p = 1$ is returned. The reason is that the bootstrap requires a minimum sample size from which to draw bootstrap samples; for example, we cannot bootstrap one data

point. Since this affects both T_{min} in Algorithm 2 and T in Algorithm 1, we use $T_{min} = T$, and ensure T is large enough by, for example, using the approximate conservative lower bound for T provided in Section 2.3. In practice, small sample sizes appear in line 6 of Algorithm 1 when fewer than $|D_0|$ points are sampled for D_1 . This happens when T_{max}/T is not a power of 2, resulting in $T_{max} - 2^t T$ leftover data points in the last iteration t . If T_{max}/T is a power of 2, then T_{min} is not required.

Algorithm 1 has three user-specified parameters: α , δ , T . In general, data use increases with smaller α (requiring smaller error), smaller δ (detecting smaller differences), and usually with larger T (more data for the initial guess). The significance level α is selected similarly to the significance level in statistical tests, which by convention have values, such as $\alpha \in \{0.1, 0.05, 0.01\}$. The tolerance δ concerns the difference between losses, and its magnitude depends on the used loss function. Typical values for the 0–1 loss may be $\delta \in \{0.1, 0.05, 0.01\}$ depending on how close the user requires the selected model to be to the true best of the k models. The initial data size T sets the minimum data use at $2T$, since T is used for finding a candidate model and T for testing it. A larger T means the first iteration uses more data, which leads to a better initial candidate model. Selecting the correct value for T is problem dependent and typically requires domain knowledge of the modeling task. A conservative lower bound for T is given by Equation (4), which is based on bounding the bootstrap approximation error. In practice, the experiments show that values, such as $T = 128$ result in the required probabilistic control for the datasets used in this paper.

2.3 | Theoretical properties of BSV

We next discuss the theoretical properties of BSV. The technical proofs are in the Appendix.

The first theorem shows that Algorithm 1 provides the probabilistic guarantee in Problem 1, when it does not run out of data:

Theorem 1. Algorithm 1 makes an error with a probability of at most α , if the loss has a finite mean and variance and the algorithm does not run out of data.

Theorem 1 relies on the fact that the p -value given by Algorithm 2 is a proper p -value for the null case of a model not in $\mathcal{B}$. The next lemma proves that this p -value is approximately proper.

Lemma 1. At iteration t , the p -value p_t approximated by Algorithm 2 is to a good accuracy a proper p -value, that is, it satisfies $\Pr(p_t < a) \lesssim a$ for any $a \in [0, 1]$, if the candidate model $i = i_C^t$ is not approximately the best, and if

the distributions of the loss values have a finite mean and variance.

The proof of Lemma 1 contains an approximate sign due to the bootstrap approximation. The following lemma shows that the bootstrap approximation converges asymptotically, at least if the losses have finite means and variances:

Lemma 2. $\Pr(S^* - \hat{S} < a')$ converges to $\Pr(\hat{S} - S < a')$ almost surely for any $a' \in [0, 1]$ at the limit of infinite data ($T \rightarrow \infty$) and bootstrap samples ($M \rightarrow \infty$), if the loss values have a finite mean and variance, where S , $\hat{S}$, and S^* are respectively the statistic of Equation (3) at the limit of infinite data, on data of size T , and on a bootstrap sample of size M .

We next provide an approximate lower bound for T in Equation (4) that ensures a good enough bootstrap approximation. The bound is derived from the observation that the highest error in the bootstrap approximation happens (at least for the 0–1 loss) when the model losses for the T data points are identical.

Observation 1. A conservative approximate lower bound for the necessary number of data points T for a good enough bootstrap approximation when using the 0–1 loss is given by:

$$T \gtrsim \frac{-\log \alpha + \log(k-1)}{\delta}. \quad (4)$$

Typically, the model losses are correlated, and a smaller value of k can be used. If the loss is instead bounded to an interval $[0, L_{\max}]$, we can replace δ with $\delta/L_{\max}$ in Equation (4).

The bound of Equation (4) shows that the number of required data points increases inversely proportionally to the required accuracy δ but only logarithmically to the required probability guarantee α . Notice that at each iteration of Algorithm 1, we double the number of data points and halve the α level at each iteration. It follows that if Equation (4) is satisfied at the first iteration of Algorithm 1, then it will also be satisfied at subsequent iterations.

The final theorem (Theorem 2) shows that, under some not-too-strict assumptions of the test statistic, the algorithm stops after a finite number of steps. The assumption is that the test statistic is sub-Gaussian, which should be true at least for any loss bounded by an interval $[0, L_{\max}]$. The non-sub-Gaussian case corresponds to a difficult learning problem for any validation set based method. Notice that there is still a non-zero probability that the algorithm does not stop before running out of validation data when there is a finite amount of validation data available.

Theorem 2. Algorithm 1 stops almost surely, at least if the distribution of the test statistic $s_i(D)$ for any model $i \in [k]$ is sub-Gaussian with variance proxy σ_i^2 upper bounded by $\sigma_i^2 \leq \sigma^2 / |D|$ for some σ , even when there is unlimited amount of data available or $T_{\max} \rightarrow \infty$.

3 | EXPERIMENTS

In the experiments, we apply BSV to synthetic (Section 3.1) and real datasets (Section 3.2). We also briefly discuss the advantages compared to CV (Section 3.3) and describe a parameter tuning use-case (Section 3.4).

The experiments demonstrate that BSV can be used as a drop-in replacement for the standard holdout approach while using fewer data and satisfying the probabilistic guarantees of the previous section. The experiments also examine how different parameters affect its performance in terms of error rate (selecting a model not in $\mathcal{B}$), actual data use, and power (how often the p -value of a correctly chosen model is below α).

The source code and the results from the experiments are available at github.com/edahelsinki/bsv. All real datasets used in the experiments are available from openml.org.

3.1 | Datasets

The datasets used in the experiments are empirical losses from given, trained models. We use both synthetically generated losses and losses on real datasets.

3.1.1 | Synthetic data

The synthetic losses are 0–1 losses generated by treating a model's loss as a Bernoulli random variable. If we denote the 0–1 loss of model i on a data point (x, y) by $L_i(x, y) = \mathbb{I}(f_i(x) = y)$, then $\Pr(L_i(x, y) = 1) = \text{loss}_i$, where $\text{loss}_i = E[L_i(x, y)]$ is the true 0–1 loss of model i . Note there are no actual “data” on which models are trained and their losses evaluated; we directly sample model losses. We generate three datasets: `synth_5`, `synth_k`, and `synth_kbest`. In `synth_5`, we generate losses for 5 models, where $\text{loss}_1 = 0.2$, $\text{loss}_2 = \text{loss}_3 = \text{loss}_4 = 0.3$, and $\text{loss}_5 = 0.4$. Correlations are also introduced between models 2 and 3, and between models 1 and 4, by randomly inserting points where they agree. In `synth_k`, we generate losses for k models where there is one clear winner, meaning model 1 is in $\mathcal{B}$, and the rest $k-1$ are not. We use $k \in \{2, 4, 8, 16, 32, 64, 128\}$, $\text{loss}_1 = \text{loss}_{\min} = 0.1$ and $\text{loss}_i = \text{loss}_{\max} = 0.8$ for $i \in [k] \setminus \{1\}$. In `synth_kbest`, we

generate losses for $k = 32$ models where there are $k_\delta = |\mathcal{B}|$ models that are in $\mathcal{B}$, and the rest $k - k_\delta$ are not. We use $k_\delta \in \{1, 2, 4, 8, 16, 32\}$, $\text{loss}_i = \text{loss}_{\min} = 0.1$ for $i \in \mathcal{B}$, and $\text{loss}_i = \text{loss}_{\max} = 0.8$ for $i \in [k] \setminus \mathcal{B}$.

3.1.2 | Real data

We again use model losses for the experiments with real datasets instead of training models and evaluating losses on data. We compute losses directly on model predictions for various datasets retrieved from the openML repository [35] using the openML R API [4]. We primarily focus on the 0–1 loss for binary classification tasks. We also compute the clipped log-loss (for models that output a probability) and the mean square error (MSE) for regression tasks. The log loss is clipped to a maximum value, $L_{\max} = 1$. The datasets were selected based on having available a large enough validation set and predictions from a large enough collection of models. The Appendix contains a list of the datasets with their openML identifiers and visualizations of model losses.

The experiments also include modified versions of the real datasets, with clear and unclear winners. These are denoted by `_clear` and `_unclear`, and are visible in Figure 6 and in the additional results in the Appendix. In the `_clear` versions of the datasets, models are removed, such that the difference between the accuracy of the best model and the remaining models is at least 0.1. In the `_unclear` versions, only the top k_{clear} performing models are kept. We use k_{clear} , the number of models in the respective `_clear` version, so that the results of the `_clear` and `_unclear` versions of a dataset are comparable and any differences are not due to the number of models.

3.2 | Experimental setup

In the experiments, Algorithm 1 is run on the data sets described above using various values for the parameters α , δ , and T .

The performance metrics are the data use n , the error rate $= \Pr(i_{\text{alg}} \notin \mathcal{B})$, and the empirical power $= \Pr(p_t \leq \alpha_t | i_{\text{alg}} \in \mathcal{B})$, where p_t and $\alpha_t = 2^{-t}\alpha$ are the p -value and the allocated α in the last iteration. Good performance means low data use while having low error and high power. Another metric shown in Table 3 is the empirical alpha $\alpha_{\text{emp}} = \Pr(p_t \leq \alpha_t | i_{\text{alg}} \notin \mathcal{B})$, which indicates that the p -values produced by the algorithm are calibrated, meaning $\alpha_{\text{emp}} \leq \alpha$. Note that the probabilistic guarantee of Algorithm 1 is $\alpha_{\text{emp}} \leq \alpha$, which is equivalent to error $\leq \alpha$ when the data use does not exceed $T_{\max}$.

The algorithm is run $n_{\text{run}} = 1000$ times for each parameter combination to compute averages, standard deviations, and empirical probabilities. Empirical probabilities are computed as follows throughout the paper,

$$\Pr(\square) = \frac{1 + \sum_{i=1}^{n_{\text{run}}} \mathbf{I}(\square_i)}{1 + n_{\text{run}}}, \quad (5)$$

where $\mathbf{I}(\square_i)$ is the indicator function which equals unity if the Boolean expression $\square_i$ is true and zero otherwise.

The experiments were run using R 4.1 [30] on a high-performance computing cluster.

3.3 | Synthetic data experiments

This section demonstrates simple scenarios using synthetic 0–1 losses for k models. Synthetic 0–1 losses are simulated as Bernoulli variables with a probability of success equal to the model's accuracy. We show that (i) the p -values computed with Algorithm 2 work as expected, (ii) there is a trade-off between error rate and data use, which BSV handles well, and (iii) BSV performs well when there are models that are superior to others.

3.3.1 | p -value simulation

We first examine the `synth_5` dataset, which contains 5 models, whose losses are shown in Figure 1. We will examine a tolerance $\delta = 0.1$, such that models 2, 3, and 4 are exactly $\delta = 0.1$ from model 1. If we now consider the p -values for these models, we expect that the p -value is often low for the best model and is often large for the non-best models, or $\Pr(p_1 \leq \alpha) \gg \alpha$, $\Pr(p_i \leq \alpha) \approx \alpha$ for $i = 2, 3, 4$, and $\Pr(p_5 \leq \alpha) \ll \alpha$.

Figure 1 shows a quantile-quantile (QQ) plot of 1000 p -values for each model (computed using the bootstrap p -values in Algorithm 2), compared against quantiles of

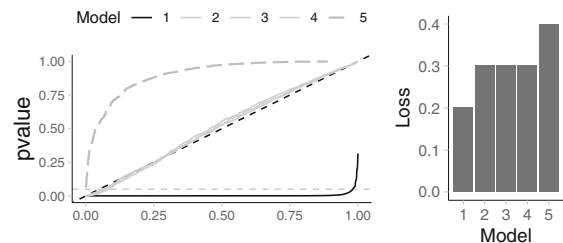


FIGURE 1 The p -values computed by Algorithm 2 are approximately proper p -values. Left: Quantile-quantile plot of a uniform distribution against 1000 p -values for each model in the `synth_5` dataset. Right: The true loss of each model.

a uniform distribution. In the QQ plot, the diagonal corresponds to a uniform distribution. We observe that the p -values behave as described in the previous section: the best model (1) is below the diagonal, meaning its p -values tend to be small (or stochastically larger than uniform), and the worst model (5) is above the diagonal, meaning its p -values tend to be large (or stochastically smaller than uniform). The middle models (2, 3, and 4) are on the diagonal, meaning their p -values are approximately uniform.

The power of the procedure (that is, how often $p_1 \leq \alpha$) can also be read from the figure for any α as the intersection of the best model with a horizontal line at α (here the dashed line is at $\alpha = 0.05$). We observe that the power is close to 1, which means the p -value for the best model is almost always less than α .

3.3.2 | Data-error tradeoff

In model selection, there is an inherent trade-off between data use and error rate, as shown in Figure 2 for the *synth_5* and *spambase* datasets. If we use n randomly sampled points to select a model with the smallest loss on this sample, then the error rate is the probability that the chosen model is not in $\mathcal{B}$. We can make this error smaller by increasing the data use n . The gray line indicates this error for a given n , which we call the reference boundary.

In BSV, we seek a solution that has an error lower than a threshold α (the dotted line denotes $\alpha = 0.05$) and that uses as little data as possible (small n). Figure 2 shows the

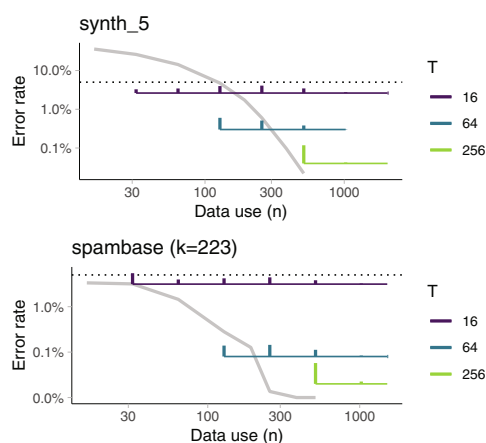


FIGURE 2 The trade-off between error and data use for the *synth_5* (top) and *spambase* (bottom) datasets. The gray curve denotes the reference boundary trade-off (described in the text). Horizontal lines denote the data use for a given error rate over 5000 runs of BSV for $T \in \{16, 64, 256\}$ and $\alpha = \delta = 0.05$. The small vertical lines denote the % of the 5000 runs in which data use was at that level.

variation in data use for a given error rate and an initial number of data points T . For smaller values of T , such as 16, data use has a large variation because BSV automatically adjusts the data use to ensure an error below $\alpha = 0.05$. The variation for larger values ($T = 256$) is smaller since the best model on T points is a good enough estimation. In both cases, BSV hovers around the reference data use and often requires less data (as indicated by the frequencies of each n , shown as vertical lines).

Notice in the *spambase* dataset that often only a few data points are required to select even between $k = 223$ models if the goal is to find a model within $\delta = 0.05$ from the best (here 142 models satisfy this requirement). While a standard train-validation split uses the whole validation data at once, BSV adaptively increases the size of the data such that the selected model is within δ from the best with an error level less than α .

3.3.3 | Effect of clear winners

We next examine how the algorithm's data use and error are affected by the number of models (k) and the number of models that are approximately the best ($k_\delta = |\mathcal{B}|$ for some δ), by running BSV on $N = 5000$ points of the *synth_k* and *synth_kbest* datasets.

In the *synth_k* dataset, there is one clear winner and $k - 1$ models that are not in $\mathcal{B}$; losses for $k = 4$ and $k = 16$ are shown in Figure 3 on the right. Figure 3 shows that in this clear winner case, increasing the number of models (larger k) does not increase the error rate (solid line, left figure) or the data use (middle figure). This indicates that the algorithm works well in the “needle in a haystack” problem of finding the single best model out of clearly inferior ones.

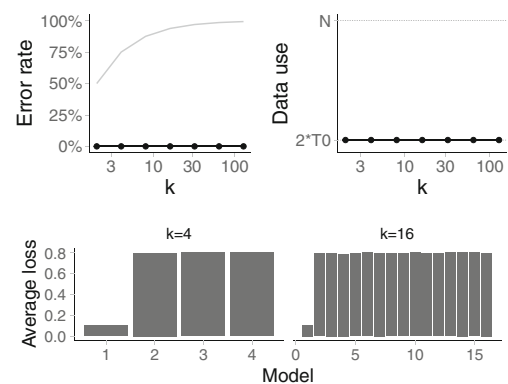


FIGURE 3 Top: Adding more “non-best” models (increasing k by adding $i \notin \mathcal{B}$) does not increase error rate and data use ($\delta = \alpha = 0.05$, $T = 128$). The gray line denotes the error when selecting a model uniformly at random (high k is difficult). Bottom: Losses for $k = 4$ and $k = 16$.

In the *synth_kbest* dataset, we vary k_δ , the number of models in $\mathcal{B}$; losses for $k_\delta = 1$ and $k_\delta = 16$ are shown in Figure 4 on the right. Notice that the data use is adjusted automatically to ensure a low error rate (middle figure). Figure 4 shows that even when there are multiple clear winners, the error rate remains constant (solid line, left figure).

The gray line in both figures denotes the error rate of a dummy model selection where a model is selected uniformly at random from k models, which leads to an error rate of $1 - k_\delta/k$. The dummy baseline highlights that model selection is challenging for many models k or for a small number of “good” models k_δ . BSV performs well in both cases.

3.4 | Real-world data experiments

We next apply BSV on real datasets, and examine how the tolerance parameter δ affect its data use.

3.4.1 | Results

The results on the real datasets demonstrate that BSV often requires only a fraction of the available data to make a selection, it can accurately select out of a large number of models k a model that is “approximately the best,” and it works using various loss functions. Table 1 summarizes the results of 1000 runs of BSV on real datasets using $\alpha = 0.05$, $T = 128$, and various values of δ . Results on additional datasets are in the Appendix.

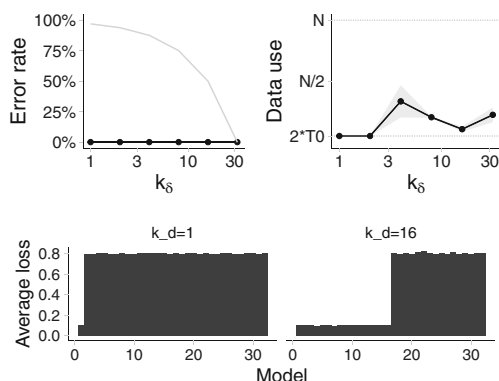


FIGURE 4 Top: Adding more “best” models (increasing $k_\delta = |\mathcal{B}|$ for $k = 32$) does not increase error rate and has a minor effect on data use ($\delta = \alpha = 0.05$, $T = 128$). The gray line denotes the error when selecting a model uniformly at random (small k_δ is difficult). Confidence bands denote a standard deviation over 1000 runs. Bottom: Losses for $k_\delta = 1$ and $k_\delta = 16$.

In the results shown in Table 1, we observe that across all datasets and losses, the average data use $E[n]$ is lower than the available validation data N , which is the data used by the standard holdout approach. We also observe that the error is controlled, meaning a correct model is selected, and the power is high, meaning the p -value is low when it should be.

In some cases, BSV requires as low as 2% of the available data on average for a model selection as accurate as the standard holdout for a given tolerance δ . These results indicate that BSV can be used as a data-efficient replacement for validation set methods. While the error is relatively constant over different scenarios, the data use and power vary, mainly due to the tolerance δ , the number of models k , and the initial number of data points T . We next look at the effect of δ in more detail.

3.4.2 | Effect of the tolerance δ

Detecting smaller differences (small δ) generally requires more data. Figure 5 illustrates how δ affects data use n for four datasets. There is some variation due to the random nature of the bootstrap, but in all datasets, the data use is less than the maximum available N , shown with a dotted line.

Especially for larger δ , model selection with BSV uses only a fraction of the available data. For example, for $\delta = 0.1$ in the *electricity* dataset, we can select between 109 models using only 267 ± 53 data points ($\approx 2\%$ of the available data), with an error of 0.1%. Generally, a smaller δ also leads to a larger error, which is nevertheless still bounded by $\alpha = 0.05$.

As was shown in the synthetic experiments, the effect of δ on data use is mediated by whether there is a clear winner, meaning a model with a loss much lower than others. We also observe this in the real datasets: when there is a clear winner, the data use is approximately constant over different values of δ (Figure 6). The error rate in these scenarios is below 1% for the clear case, while for the unclear case it is zero when all models are approximately the best since any selection is correct.

3.5 | Comparison to cross validation

One advantage of BSV over CV is that the models need to be trained only once. In contrast, CV requires models to be trained once per fold, which, at the limit of leave-one-out CV (LOOCV), is equal to the number of data points. Table 2 compares the number of times k models are trained when using BSV versus when using CV and LOOCV.

TABLE 1 Results of running BSV on real datasets, using $\alpha = 0.05$, $T = 128$, and various δ

Loss	Dataset	N	$k(k_\delta)$	δ	$E[n](sd(n))$	$E[n]/N$	error	power _{emp}
0-1	adult	16,117	103 (34)	0.01	6977 (5085)	43%	0.7%	98.7%
0-1	adult	16,117	103 (91)	0.1	280 (74)	2%	0.1%	100.0%
0-1	BNG (breast-w)	12,990	108 (77)	0.01	1254 (1150)	10%	0.7%	99.4%
0-1	BNG (breast-w)	12,990	108 (103)	0.1	257 (14)	2%	0.1%	100.0%
0-1	BNG (tic-tac-toe)	12,990	111 (19)	0.01	9484 (4343)	73%	0.5%	89.2%
0-1	BNG (tic-tac-toe)	12,990	111 (95)	0.1	298 (103)	2%	0.1%	100.0%
0-1	electricity	14,952	109 (8)	0.01	2477 (2531)	17%	0.9%	99.2%
0-1	electricity	14,952	109 (44)	0.1	271 (63)	2%	0.1%	100.0%
0-1	kr-vs-kp	1054	226 (85)	0.01	525 (288)	50%	0.6%	98.9%
0-1	kr-vs-kp	1054	226 (192)	0.1	256 (8)	24%	0.1%	100.0%
0-1	mushroom	2680	215 (176)	0.01	262 (39)	10%	0.6%	99.5%
0-1	mushroom	2680	215 (207)	0.1	256 (0)	10%	0.1%	100.0%
0-1	sick	1244	204 (55)	0.01	758 (372)	61%	0.7%	97.2%
0-1	sick	1244	204 (203)	0.1	257 (11)	21%	0.1%	100.0%
0-1	spambase	1518	223 (39)	0.01	1138 (445)	75%	0.7%	79.8%
0-1	spambase	1518	223 (180)	0.1	262 (40)	17%	0.1%	100.0%
MSE	cpu_small	820	4 (1)	0.1	478 (217)	58%	0.1%	99.9%
MSE	cpu_small	820	4 (2)	5	256 (0)	31%	0.1%	100.0%
MSE	delta_elevators	952	7 (5)	1 e-06	259 (27)	27%	0.1%	100.0%
MSE	delta_elevators	952	7 (2)	2 e-07	441 (239)	46%	0.1%	99.8%
MSE	kin8nm	820	6 (3)	0.001	571 (248)	70%	0.1%	92.4%
MSE	kin8nm	820	6 (5)	0.01	259 (28)	32%	0.1%	100.0%
MSE	topo_2_1	889	5 (4)	0.0002	313 (122)	35%	0.1%	100.0%
MSE	topo_2_1	889	5 (2)	8 e-05	586 (285)	66%	0.4%	77.4%
LL	adult_ll	16,117	54 (14)	0.01	688 (556)	4%	1.2%	98.9%
LL	adult_ll	16,117	54 (50)	0.1	259 (29)	2%	0.1%	100.0%
LL	BNG (breast-w)_ll	12,990	61 (37)	0.01	522 (376)	4%	0.5%	99.6%
LL	BNG (breast-w)_ll	12,990	61 (56)	0.1	256 (8)	2%	0.1%	100.0%
LL	BNG (tic-tac-toe)_ll	12,990	78 (17)	0.01	1938 (1879)	15%	0.9%	99.2%
LL	BNG (tic-tac-toe)_ll	12,990	78 (76)	0.1	259 (25)	2%	0.1%	100.0%
LL	electricity_ll	14,952	71 (8)	0.01	340 (165)	2%	0.4%	99.7%
LL	electricity_ll	14,952	71 (36)	0.1	256 (0)	2%	0.1%	100.0%
LL	kr-vs-kp_ll	1054	149 (26)	0.01	404 (225)	38%	0.1%	100.0%
LL	kr-vs-kp_ll	1054	149 (116)	0.1	256 (0)	24%	0.1%	100.0%
LL	mushroom_ll	2680	203 (165)	0.01	256 (0)	10%	0.1%	100.0%
LL	mushroom_ll	2680	203 (196)	0.1	256 (0)	10%	0.1%	100.0%
LL	sick_ll	1244	111 (22)	0.01	564 (344)	45%	0.2%	98.8%
LL	sick_ll	1244	111 (109)	0.1	256 (0)	21%	0.1%	100.0%
LL	spambase_ll	1518	110 (7)	0.01	671 (379)	44%	2.8%	97.3%
LL	spambase_ll	1518	110 (87)	0.1	258 (23)	17%	0.1%	100.0%

Note: The columns are N : number of available validation data points, k : number of models, k_δ : number of models in $\mathcal{B}$ for a given δ , δ : tolerance parameter, $E[n]$: average data use, $sd(n)$: standard deviation of data use, error: proportion of incorrect selections, power_{emp}: proportion of correct selections with a final p -value less than the required threshold. Averages and proportions are over 1000 runs of BSV.

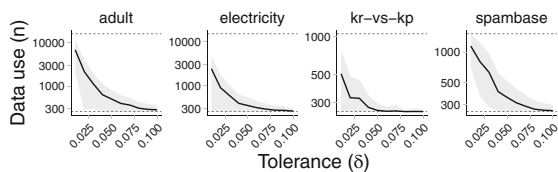


FIGURE 5 The effect of the tolerance parameter (δ) on data use (n) using 0–1 loss. Detecting smaller differences (smaller δ) requires more data. The data used is a fraction of the total available data for larger differences. The upper dotted horizontal line denotes the maximum available data for validation for each dataset (N), and the lower dotted line denotes the minimum data used for the particular T ($2T$). Confidence bands denote standard deviation over 1000 runs with $\alpha = 0.05$ and $T = 128$.

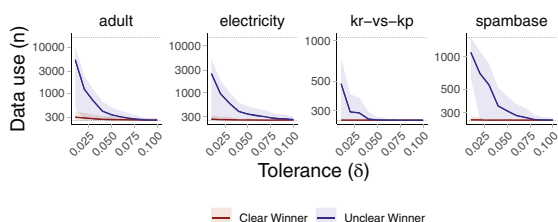


FIGURE 6 The effect of the tolerance parameter (δ) on data use (n), when there is a clear winner ($B = 1$; only one model is within δ from the best) and when there is no clear winner ($B = k$; all models are within δ from the best). Less data is used when there is a clear winner. Confidence bands denote standard deviation over 1000 runs with $\alpha = 0.05$ and $T = 128$.

In a typical use of BSV, N_{tr} points are used for training and $n < N_{va}$ points from a validation set are used for model selection. In contrast, CV with 10 folds uses $N = N_{tr} + N_{va}$ points in total and $N/10$ points per fold for model selection.

3.6 | Parameter tuning use case

So far, we have considered model selection when given k models. Another common use case in model building is parameter tuning. In parameter tuning, we seek the optimal parameters for a single model. We can perform model selection in continuous spaces by defining a grid over the parameter values. Parameter tuning can then be viewed as a model selection task over k different model versions.

We here present an example of tuning the regularization parameter λ in LASSO regression on the spambase dataset. We first train the LASSO model (using $N_{tr} = 460$ points) on a grid of $k = 95$ values for λ , producing k versions of the model, and then use BSV (on $N_{va} = 461$ validation points) to select a model that is approximately the best. The top images in Figure 7 show the test losses (on $N_{te} = 3680$

TABLE 2 Number of times k models are trained when using different validation methods. BSV requires models to be trained only once.

k	BSV	10-fold CV	LOOCV, $N = 100$
2	2	20	200
4	4	40	400
8	8	80	800
16	16	160	1600
32	32	320	3200
64	64	640	6400
128	128	1280	12,800
256	256	2560	25,600
512	512	5120	51,200
1024	1024	10,240	102,400

points) for a LASSO regression with each of the $k = 95$ values of λ . The bottom figures show how often each λ was selected by BSV over 1000 trials. We observe that the models selected by BSV are mostly within the boundaries of B , shown with vertical dotted lines.

Typically, when CV is used in parameter tuning, it estimates the generalization error and selects the λ_{min} with the lowest CV estimate of the error. In Figure 7, the shown λ_{min} is computed by the `glmnet` R package [12]. As mentioned in the previous section, 10-fold CV offers a lower error rate than BSV, but uses more data ($N = N_{va} + N_{tr} = 921$ points), and trains each model 10 times.

The advantage of BSV is that it allows parameter tuning to trade-off between data use and error rate by adjusting the tolerance δ . For example, for $\delta = 0.1$ in Table 3 the error rate is 0.1%, and the data use $E[n]$ can be a fraction of the whole validation data, while for $\delta = 0.01$ we get a 12%–15% error rate if T is too small.

Notice that in Table 3 the error rate is sometimes larger than α . This occurs if Algorithm 1 has not yet confirmed the candidate model is correct (the p -value is not below the allocated α level) when it runs out of data (T_{max} is reached). In these cases, the algorithm outputs a warning.

4 | DISCUSSION

We have presented the BSV algorithm that can find a “good enough” classification or regression model out of a collection of given models by adaptively changing the validation set size. In BSV, the user can pre-define a probability and a tolerance, such that the algorithm is guaranteed with a high probability to output a model that is “approximately the best,” that is, a model with an expected loss on new

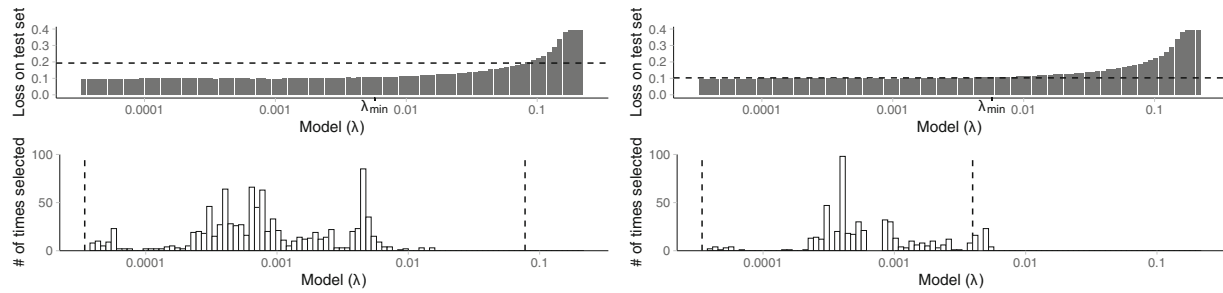


FIGURE 7 Parameter tuning example with λ in LASSO regression on the spambase dataset. Above: Test set losses of LASSO regressors with a regularization parameter λ . Bottom: How often BSV selects a particular λ . Left: BSV with $\delta = 0.1$, $T = 128$, $\alpha = 0.1$. Right: BSV with $\delta = 0.01$, $T = 128$, $\alpha = 0.05$. The value with minimum CV loss is denoted by $\lambda_{\min}$. Models within the dotted lines are approximately the best.

TABLE 3 Parameter tuning for LASSO regression on the spambase dataset using BSV with $\alpha = 0.05$. See Table 1 for column descriptions.

δ	T	$E[n](sd(n))$	$E[n]/N$	error	α_{emp}	$\text{power}_{\text{emp}}$
0.1	64	138 (37)	30%	0.1%	0.1%	100.0%
0.1	128	258 (19)	56%	0.1%	0.1%	100.0%
0.1	256	460 (0)	100%	0.1%	0.1%	100.0%
0.01	64	345 (148)	75%	14.2%	4.4%	63.1%
0.01	128	424 (79)	92%	14.2%	1.4%	54.2%
0.01	256	461 (0)	100%	0.1%	0.1%	45.3%

data that is within the given tolerance from the smallest loss.

We have shown theoretically and experimentally that BSV uses data economically and that it can be used as a drop-in replacement for validation methods for black box supervised learning models. In addition to optimizing the usage of validation data, the advantages of BSV include a statistical guarantee for the selected model and that training the models only once is sufficient, as in a simple training-validation set split. The standard CV methods may still be preferable if a statistical guarantee is not required, training the models several times (one for each CV fold) is acceptable, and the validation data is not needed later.

We see a few directions for future work. First, the bootstrap p -value computed by Algorithm 2 is based on the basic bootstrap confidence interval, but it can be based on a different type of bootstrap confidence interval. Second, in Algorithm 1, the total error rate α allocated to each iteration t is $2^{-t}\alpha$ while the data use is increased as $2^t T$, where T is the initial data size. Other options besides halving the error and doubling the data are possible and may result in improved performance. Third, BSV currently assumes i.i.d. data; therefore, it does not, in theory, handle dependent data, such as time series. That said, the electricity dataset used in the experiments is a time

series, so in practice, the approach may work for dependent data under certain conditions. Extensions for dependent data may include using a bootstrap method that handles dependent data and modifying the iterative testing procedure to utilize known data autocorrelations for improved power. BSV can potentially be used in applications outlined in Section 1.1, such as concept drift detection and iterative data exploration.

Finally, even though in most examples discussed in this paper, the k models are from different model families, Problem 1 is a formulation of a generic learning problem. The models f_j , where $j \in [k]$, can also present a model family parametrized by a discretized parameter vector j . For example, the model family could be a given deep learning architecture, and the parameter vector j would consist of the network weights. Solving Problem 1 would then be equivalent to finding the model parameters that provide the best generalization performance. While a naive application of BSV to such a task would be infeasible (except for limited use cases, such as parameter tuning in Section 3.4), the same ideas could be used to formulate a variant of stochastic gradient descent (SGD). One iteration of this SGD variant would correspond to using (a modified) BSV to select a best-performing parameter vector from a neighborhood of the best-performing parameter vector found so far.

ACKNOWLEDGMENTS

The authors thank the Finnish Computing Competence Infrastructure (FCCI) for supporting this project with computational and data storage resources. We acknowledge the support of the Doctoral Programme in Computer Science at the University of Helsinki, Academy of Finland (decisions 345704 and 346376), and the Future Makers Program of the Technology Industries of Finland Centennial Foundation and Jane and Aatos Erkko Foundation. Open Access funding enabled and organized by the Helsinki University Library.

DATA AVAILABILITY STATEMENT

The source code to reproduce the experimental results are publicly available at github.com/edahelsinki/bsv. The datasets used in the experiments are publicly available at openml.org.

ORCID

Rafael Savvides  <https://orcid.org/0000-0002-5796-6274>

Jarmo Mäkelä  <https://orcid.org/0000-0002-8788-3939>

Kai Puolamäki  <https://orcid.org/0000-0003-1819-1047>

REFERENCES

1. S. Arlot and A. Celisse, *A survey of cross-validation procedures for model selection*, Stat. Surv. 4 (2010), 40–79.
2. P. C. Austin and J. V. Tu, *Bootstrap methods for developing predictive models*, Am. Stat. 58 (2004), 131–137.
3. Y. Bengio and Y. Grandvalet, *No unbiased estimator of the variance of k-fold cross-validation*, J. Mach. Learn. Res. 5 (2004), 1089–1105.
4. G. Casalicchio, J. Bossek, M. Lang, D. Kirchhoff, P. Kerschke, B. Hofner, H. Seibold, J. Vanschoren, and B. Bischl, *OpenML: An R package to connect to the machine learning platform OpenML*, Comput. Stat. 32 (2017), no. 3, 1–15.
5. F. Concas, J. Mineraud, E. Lagerspetz, S. Varjonen, X. Liu, K. Puolamäki, P. Nurmi, and S. Tarkoma, *Low-cost outdoor air quality monitoring and sensor calibration: A survey and critical analysis*, ACM Trans. Sensor Networks 17 (2021), no. 2, 1–44.
6. J. Demšar, *Statistical comparisons of classifiers over multiple data sets*, J. Mach. Learn. Res. 7 (2006), 1–30.
7. T. G. Dietterich, *Approximate statistical tests for comparing supervised classification learning algorithms*, Neural Comput. 10 (1998), no. 7, 1895–1923.
8. J. Ding, V. Tarokh, and Y. Yang, *Model selection techniques: An overview*, IEEE Signal Process. Mag. 35 (2018), no. 6, 16–34.
9. Cynthia Dwork, Vitaly Feldman, Moritz Hardt, Toniann Pitassi, Omer Reingold, and Aaron Leon Roth, *Preserving statistical validity in adaptive data analysis*. In Proceedings of the forty-seventh annual ACM symposium on theory of computing, pages 117–126, 2015.
10. B. Efron, *Estimation and accuracy after model selection*, J. Am. Stat. Assoc. 109 (2014), 991–1007.
11. Vitaly Feldman and Thomas Steinke, *Calibrating noise to variance in adaptive data analysis* In Conference on learning theory, pages 535–544. PMLR, 2018.
12. J. Friedman, T. Hastie, and R. Tibshirani, *Regularization paths for generalized linear models via coordinate descent*, J. Stat. Softw. 33 (2010), no. 1, 1–22.
13. J. Gama, I. Žliobait, A. Bifet, M. Pechenizkiy, and A. Bouchachia, *A survey on concept drift adaptation*, ACM Comput. Surv. 46 (2014), no. 4, 1–37.
14. T. Hastie, R. Tibshirani, and J. Friedman, *The elements of statistical learning: Data mining, inference, and prediction*, 2nd ed., Springer, New York, 2009.
15. B. Kang, K. Puolamäki, J. Lijffijt, and T. de Bie, *A constrained randomization approach to interactive visual data exploration with subjective feedback*, IEEE Trans. Knowl. Data Eng. 32 (2020), no. 9, 1666–1679.
16. Ron Kohavi, *A study of cross-validation and bootstrap for accuracy estimation and model selection* In Proceedings of the 14th IJCAI, pages 1137–1143, 1995.
17. S. Konishi and G. Kitagawa, *Generalised information criteria in model selection*, Biometrika 83 (1996), no. 4, 875–890.
18. T. Krueger, D. Panknin, and M. Braun, *Fast cross-validation via sequential testing*, J. Mach. Learn. Res. 16 (2015), no. 33, 1103–1155.
19. L. Li, K. Jamieson, G. DeSalvo, A. Rostamizadeh, and A. Talwalkar, *Hyperband: A novel bandit-based approach to hyperparameter optimization*, J. Mach. Learn. Res. 18 (2017), no. 1, 6765–6816.
20. Thomas Liao, Rohan Taori, Deborah Raji, and Ludwig Schmidt, *Are we learning yet? A meta review of evaluation failures across machine learning* In J. Vanschoren and S. Yeung, editors, Proceedings of the neural information processing systems track on datasets and benchmarks, volume 1, 2021.
21. Oded Maron and Andrew W. Moore, *Hoeffding races: Accelerating model selection search for classification and function approximation* In Proceedings of the 6th NIPS, pages 59–66, 1993.
22. O. Maron and A. W. Moore, *The racing algorithm: Model selection for lazy learners*, Artif. Intell. Rev. 11 (1997), no. 1, 193–225.
23. Charles Marx, Flavio Calmon, and Berk Ustun, *Predictive multiplicity in classification* In International conference on machine learning, pages 6765–6774. PMLR, 2020.
24. B. V. North, D. Curtis, and P. C. Sham, *A note on the calculation of empirical p values from Monte Carlo procedures*, Am. J. Hum. Genet. 72 (2002), no. 2, 498–499.
25. E. Oikarinen, H. Tiittanen, A. Henelius, and K. Puolamäki, *Detecting virtual concept drift of regressors without ground truth values*, Data Min. Knowl. Disc. 35 (2021), 726–747.
26. S. J. Pocock, *Group sequential methods in the design and analysis of clinical trials*, Biometrika 64 (1977), no. 2, 191–199.
27. Lutz Prechelt, *Early stopping-but when?* In Neural networks: Tricks of the trade, pages 55–69. 1998.
28. K. Puolamäki, E. Oikarinen, and A. Henelius, *Guided visual exploration of relations in data sets*, J. Mach. Learn. Res. 22 (2021), no. 96, 1–32.
29. K. Puolamäki, E. Oikarinen, B. Kang, J. Lijffijt, and T. de Bie, *Interactive visual data exploration with subjective feedback: An information-theoretic approach*, Data Min. Knowl. Disc. 34 (2020), no. 1, 21–49.
30. R Core Team, *R: A language and environment for statistical computing*, R Foundation for Statistical Computing, Vienna, Austria, 2021.

31. Russo Daniel and Zou James, *How much does your data exploration overfit? Controlling bias via information usage*, IEEE Transactions on Information Theory 66 (2020), no. 1, 302–323. <https://doi.org/10.1109/TIT.2019.2945779>
32. Rafael Savvides, Andreas Henelius, Emilia Oikarinen, and Kai Puolamäki, Significance of patterns in data visualisations In Proceedings of the 25th KDD, pages 1509–1517, 2019.
33. L. Semenova, C. Rudin, and R. Parr, On the Existence of Simpler Machine Learning Models. 2022 ACM Conference on Fairness, Accountability, and Transparency, FAccT '22 Association for Computing Machinery, New York, 2022, 1827–1858.
34. J. Shao, *Bootstrap model selection*, J. Am. Stat. Assoc. 91 (1996), no. 434, 655–665.
35. J. Vanschoren, J. N. van Rijn, B. Bischl, and L. Torgo, *OpenML: Networked science in machine learning*, SIGKDD Explor. 15 (2013), no. 2, 49–60.
36. B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*, Chapman and Hall, 1993.
37. Cosma Rohilla Shalizi. *Advanced Data Analysis from an Elementary Point of View*. 2021. Downloaded on 7 December 2021.
38. J. Shao and T. Dongsheng, *The Jackknife and Bootstrap*. Springer Series in Statistics, Springer, New York Heidelberg, 2., corr. print edition, 1996.

How to cite this article: R. Savvides, J. Mäkelä, and K. Puolamäki, *Model selection with bootstrap validation*, Stat. Anal. Data Min.: ASA Data Sci. J. **16** (2023), 162–186. <https://doi.org/10.1002/sam.11606>

APPENDIX A

A.1 Technical proofs

The proofs based on the bootstrap are true asymptotically at the limit $T \rightarrow \infty$ if the loss has a finite and well-defined mean and variance. Also, the proofs assume that we can compute p -values p_t , which are approximated in Algorithm 2 (line 6) with an empirical p -value $\hat{p}_t(M)$ on M bootstrap samples. By the law of large numbers $\hat{p}_t(M) \rightarrow p_t$ as $M \rightarrow \infty$. Therefore, the proof holds asymptotically for $\hat{p}_t(M)$ at the limit $M \rightarrow \infty$.

Proof of Theorem 1. Algorithm 1 makes an error—assuming the data limit T_{max} is not reached—only if it outputs a model i that is not approximately the best or $i \notin B$. The error can happen in iteration t only if the candidate model i_C^t is not approximately the best and the p -value p_t satisfies the termination condition $p_t \leq 2^{-t}\alpha$. Below, we analyze the infinite algorithm that does an infinite number of iterations and only outputs approximately the best model at iteration τ (where Algorithm 1 would terminate). Algorithm 1 makes an error only if the

infinite algorithm has at least one iteration t where $i_C^t \notin B$ and $p_t \leq 2^{-t}\alpha$. The probability of at least one such event is upper bounded by:

$$\begin{aligned}
 & \Pr(\text{Alg. 1 makes an error}) \\
 & \leq \Pr\left(\bigcup_{t=1}^{\infty} (i_C^t \notin B \wedge p_t \leq 2^{-t}\alpha)\right) \\
 & \leq \sum_{t=1}^{\infty} \Pr(i_C^t \notin B \wedge p_t \leq 2^{-t}\alpha) \\
 & \leq \sum_{t=1}^{\infty} \Pr(p_t \leq 2^{-t}\alpha \mid i_C^t \notin B) \\
 & \lesssim \sum_{t=1}^{\infty} 2^{-t}\alpha = \alpha.
 \end{aligned} \tag{A1}$$

The first inequality follows from the observation that if Algorithm 1 makes an error, then the infinite algorithm must have at least one iteration where $i_C^t \notin B$ and $p_t \leq 2^{-t}\alpha$. The last inequality follows from Lemma 1 below. $\square$

Proof of Lemma 1. We use the bootstrap to study the statistic $s_i(D)$ given by:

$$\begin{aligned}
 s_i(D) &= L_i(D) - \min_{i' \in [k] \setminus \{i\}} L_{i'}(D) \\
 &= \max_{i' \in [k] \setminus \{i\}} (L_i(D) - L_{i'}(D)),
 \end{aligned} \tag{A2}$$

which is an approximator for the asymptotic “true” value denoted here by $S = \lim_{|D| \rightarrow \infty} s_i(D)$. In this proof, we take i to be fixed and assume that the model i is not approximately the best, which implies that $\delta < S$. Further, we assume we are given a constant $a \in [0, 1]$.

We denote by D^* a random variable representing a bootstrap sample of the dataset D , obtained by sampling $|D|$ data points from D randomly with replacement. We denote the corresponding test statistic by $S^* = s_i(D^*)$, and by $Q(a)$ the a th quantile of the bootstrap distribution of S^* . Furthermore, we use $\hat{S} = s_i(D)$ to denote the value of the statistic on the original (non-bootstrapped) data.

We can write:

$$\begin{aligned}
 a &= \Pr(S^* < Q(a)) \\
 &= \Pr(S^* - \hat{S} < Q(a) - \hat{S}) \\
 &\approx \Pr(\hat{S} - S < Q(a) - \hat{S}) \\
 &= \Pr(2\hat{S} - Q(a) < S) \\
 &= \Pr(\Delta_a < S),
 \end{aligned} \tag{A3}$$

where we defined a new random variable $\Delta_a = 2\hat{S} - Q(a)$ for convenience. The first equality follows from the

definition of the a th quantile. The approximate sign follows from the assumption that the distribution of the random variable $S^* - \hat{S}$ converges to $\hat{S} - S$ almost surely at the limit $T \rightarrow \infty$, at least if the loss has a finite mean and variance. For proof of convergence, we refer to Lemma 2 below.

Notice that Equation (A3) gives the level- α basic bootstrap upper bound for the value of S [36, 37]²; see Sect. 6.2.3 of [37] for discussion and analogous derivations.

From $\delta < S$ (which follows from our assumption that the model i was not approximately the best) and Equation (A3), it follows that:

$$\Pr(\Delta_a < \delta) < \Pr(\Delta_a < S) \approx a. \quad (\text{A4})$$

By rearranging $\Delta_a = 2\hat{S} - Q(a) < \delta$ we obtain an equivalent expression $p_t = Q^{-1}(2\hat{S} - \delta) < a$, where we have defined a new random variable $p_t = Q^{-1}(2\hat{S} - \delta)$. p_t is the p -value computed approximately by Algorithm 2 (line 6). Finally, by replacing $\Delta_a < \delta$ with $p_t < a$ in Equation (A4), we end up with the expression:

$$\Pr(p_t < a) \lesssim a, \quad (\text{A5})$$

which proves the claim. $\square$

Proof of Lemma 2. We refer to Example 3.1 of [38], which shows the consistency of the bootstrap for the sample mean of random p -vectors with a finite mean and variance.

The p -vectors of [38], denoted by X , correspond to $k - 1$ -dimensional vectors ($p = k - 1$), where each of the elements (indexed by j) is given by the difference in losses, or $X_j = L(f_i(x), y) - L(f_j(x), y)$. The cumulative distribution of X is defined by $H_T(a') = \Pr(X_1 \leq a' \wedge \dots \wedge X_{k-1} \leq a') = \Pr(\max_j X_j \leq a')$, where we have taken $i = k$ for simplicity of notation. $H_{BOOT}(a')$ can be defined analogously.

With these definitions, [38] shows that the bootstrap estimator H_{BOOT} converges almost surely to the cumulative distribution H_T at the limit of infinite data ($T \rightarrow \infty$) and bootstrap samples ($M \rightarrow \infty$) if the random variables (or here the losses) have a finite mean and variance. $\square$

Proof of Theorem 2. In the proof, we are interested only in the asymptotic behavior of the algorithm when $t \rightarrow \infty$. At this limit, the candidate model $i = \lim_{t \rightarrow \infty} i_C^t$ is the model with the smallest loss with a very high probability, since $|D| \propto 2^t$. In the remainder of this proof, we, therefore, assume that $\text{loss}_i = \text{loss}_{\text{best}}$. The Chernoff bound for sub-Gaussian random variables reads:

$$\begin{aligned} \Pr(s_i(D) > \Delta) &\leq \exp\left(-\frac{\Delta^2}{2\sigma_i^2}\right) \\ &\leq \exp\left(-|D|\frac{\Delta^2}{2\sigma^2}\right). \end{aligned} \quad (\text{A6})$$

The left-hand side of the equation is a probability that acts as a proxy for the p -value of the best model i , for some constant Δ . The algorithm stops if the p -value goes below $2^{-t}\alpha$. The right-hand side is proportional to $\exp(-\lambda|D|) \propto \exp(-2^t\lambda)$ for some $\lambda > 0$. This upper bound goes down faster than the algorithm stopping limit, and therefore Algorithm 1 stops after a finite number of steps almost surely. $\square$

A.1.1. Derivation of Observation 1

We first introduce additional notation and establish an auxiliary lemma. Consider the case of a 0–1 loss with two models ($k = 2$), and take $i = 1$ in (2). The test statistic of Equation (A2) is then given by $\hat{S} = \frac{1}{T} \sum_{i=1}^T Z_i$, where T is the sample size and $Z_i = L(f_1(x_i), y_i) - L(f_2(x_i), y_i) \in \{-1, 0, 1\}$. Denote the probability of $Z_i = -1$ by p_- (model 1 makes a wrong prediction and model 2 gives a correct prediction), the probability of $Z_i = 0$ by p_0 (models give the same prediction), and the probability of $Z_i = 1$ by p_+ (model 1 makes a correct and model 2 a wrong prediction). The “true” value of the test statistic (at the limit $T \rightarrow \infty$)—here, the difference in model accuracies—is given by $S = p_+ - p_-$. Denote by x_- , x_0 , and x_+ the counts of observed values -1 , 0 , and 1 of Z_i , respectively. Notice that $p_- + p_0 + p_+ = 1$ and $x_- + x_0 + x_+ = T$ by definition.

We can use the observed counts to compute the level- a' exact upper bound Δ_E for the value of S as follows.

Lemma 3. Given $a' \in [0, 1]$, the probability p_- , and the observed counts (x_-, x_0, x_+) , the level- a' exact upper bound Δ_E for the value of S is given by:

$$\Delta_E = -p_- + (1 - p_-) \times B(1 - a'; x_+ + 1, x_0 + 1), \quad (\text{A7})$$

where $B(p; v, w)$ is the p th quantile from Beta distribution with shape parameters v and w and where we have assumed uniform prior $\Pr(S|p_-)$ for $S \in [-p_-, 1 - p_-]$.

Proof. The trinomial distribution gives the likelihood of the observed counts:

$$\Pr(x_-, x_+ | p_-, p_+) = \frac{T!}{x_-! x_0! x_+!} p_-^{x_-} p_0^{x_0} p_+^{x_+}, \quad (\text{A8})$$

²The value of S is within probability $1 - a$ in the interval $[-\infty, \Delta_a]$.

where $x_0 = T - x_- - x_+$ and $p_0 = 1 - p_- - p_+$. We can re-parametrize the likelihood by S and p_- :

$$\Pr(x_-, x_+ | S, p_-) = \frac{T!}{x_-! x_0! x_+!} p_-^{x_-} (1 - S - 2p_-)^{x_0} (S + p_-)^{x_+}. \quad (\text{A9})$$

Due to the uniform prior $\Pr(S|p_-)$ for $S \in [-p_-, 1 - p_-]$, we can write the posterior for S (where the normalization term Z contains terms constant in S) as:

$$\Pr(S | x_-, x_+, p_-) = Z^{-1} (1 - S - 2p_-)^{x_0} (S + p_-)^{x_+}. \quad (\text{A10})$$

The level- a' exact upper bound $\Delta_E \in [-p_-, 1 - p_-]$ for S is defined by:

$$\int_{-p_-}^{\Delta_E} \Pr(S | x_-, x_+, p_-) dS = 1 - a'. \quad (\text{A11})$$

The claim follows directly from the definition of the Beta distribution after inserting the expression of the posterior from Equation (A10) into the integral of Equation (A11). $\square$

Observation 1. We can now derive a conservative approximate lower bound for T . Equation (A3) essentially tries to estimate the upper bound Δ_E by using the bootstrap approximation, denoted by Δ_a . In practice, the largest error $\delta' = \Delta_E - \Delta_a$ in the upper bound is obtained when $x_0 = T$ and $x_- = x_+ = 0$, that is, the two models give the same predictions for the T data points. If this happens, the bootstrap approximation gives $\Delta_a = 0$, and therefore the bootstrap error is upper bounded by:

$$\begin{aligned} \delta' = \Delta_E &= -p_- + (1 - p_-) \times B(1 - a'; 1, T) \\ &\leq B(1 - a'; 1, T), \end{aligned} \quad (\text{A12})$$

which applies to any value of $p_- \in [0, 1]$, which allows us to drop the nuisance parameter p_- from consideration, and where we have assumed that $x_- = x_+ = 0$. We can rewrite Equation (A12) as:

$$1 - a' \leq \int_0^{\delta'} (T + 1)(1 - S)^T dS = 1 - (1 - \delta')^{T+1}. \quad (\text{A13})$$

Solving for T , we obtain:

$$T \geq \frac{\log a'}{\log(1 - \delta')} - 1 \approx -\frac{\log a'}{\delta'}, \quad (\text{A14})$$

where we have used $\log(1 - \delta') \approx -\delta'$. The lower bound for the case $k \geq 3$ follows by simply noticing that the test

statistic S is a maximum of $k - 1$ differences for which we can make a Bonferroni correction for a' or:

$$T \geq \frac{\log(a'/(k - 1))}{\log(1 - \delta')} - 1 \approx \frac{-\log a' + \log(k - 1)}{\delta'}. \quad (\text{A15})$$

It, therefore, follows that if we want the bootstrap error δ' to be smaller than the tolerance parameter δ or $\delta' \lesssim \delta$ with a probability of at least $1 - \alpha$ or $a' \lesssim \alpha$, we can, by setting $\delta' \rightarrow \delta$ and $a' \rightarrow \alpha$, obtain a conservative approximate lower bound for the number of data points:

$$T \gtrsim \frac{-\log \alpha + \log(k - 1)}{\delta}. \quad (\text{A16})$$

If the number of bootstrap samples T satisfies Equation (A16), then we expect the bootstrap approximation to behave sufficiently nicely.

A.2 Dataset descriptions

Table A1 contains the OpenML [35] identifiers for the datasets used in the experiments. Data ID refers to a dataset, and Task ID refers to a dataset plus an estimation procedure, either “33% Holdout Set” or “10-fold Cross Validation.” Note that the experiments use model predictions on these datasets but not the datasets themselves (except for the parameter tuning example). The log-loss data denoted with `_ll` in the experiments are computed on the same datasets as the ones with the 0–1 loss.

The openML datasets were selected as follows: For the binary classification tasks, we retrieved predictions on a 1/3 holdout set (reported as N in the tables) from models trained on the other 2/3 of the data. The selection criteria were that the datasets should have at least 3000 data points (of which 1/3 are in the validation set) and predictions from at least 10 models. For binary classification, we also retrieved 10-fold cross-validated model predictions. We used the first fold as the validation set (also reported as N in the tables) for models trained on the other folds (9/10 of the data). For these datasets, the selection criteria were a sample size of 10^4 – 10^6 data points and predictions from 10 to 500 models. The clipped log-loss was computed for a subset of the binary classification models that output a probability. For regression tasks, due to the lack of holdout-set predictions in openML that satisfy the above criteria, we only retrieved 10-fold cross-validated model predictions, as described above. The selection criteria for regression were that the datasets should have at least 1000 points and predictions from at least five models.

Figures A1 and A2 summarize the empirical model losses on the validation set for the selected datasets. The shape of the “loss profile” shown in the bar chart summarizes the properties of the model selection task, such as whether there are clear or unclear winners.

A.3 Additional results

Additional experimental results on more parameter value combinations and datasets are shown in Tables A2 (0–1 loss), A3 (0–1 loss for clear and unclear cases), A4 and A5 (0–1 loss with additional datasets), A6 (log loss), and A7 (MSE loss).

The columns in the tables are N : number of available validation data points, k : number of models, k_δ : number

of models in $\mathcal{B}$ for a given δ , δ : tolerance parameter, $E[n]$: average data use, $sd(n)$: standard deviation of data use, error: proportion of incorrect selections, $power_{emp}$: proportion of correct selections with a final p -value less than the required threshold. Averages and proportions are over 1000 runs of BSV.

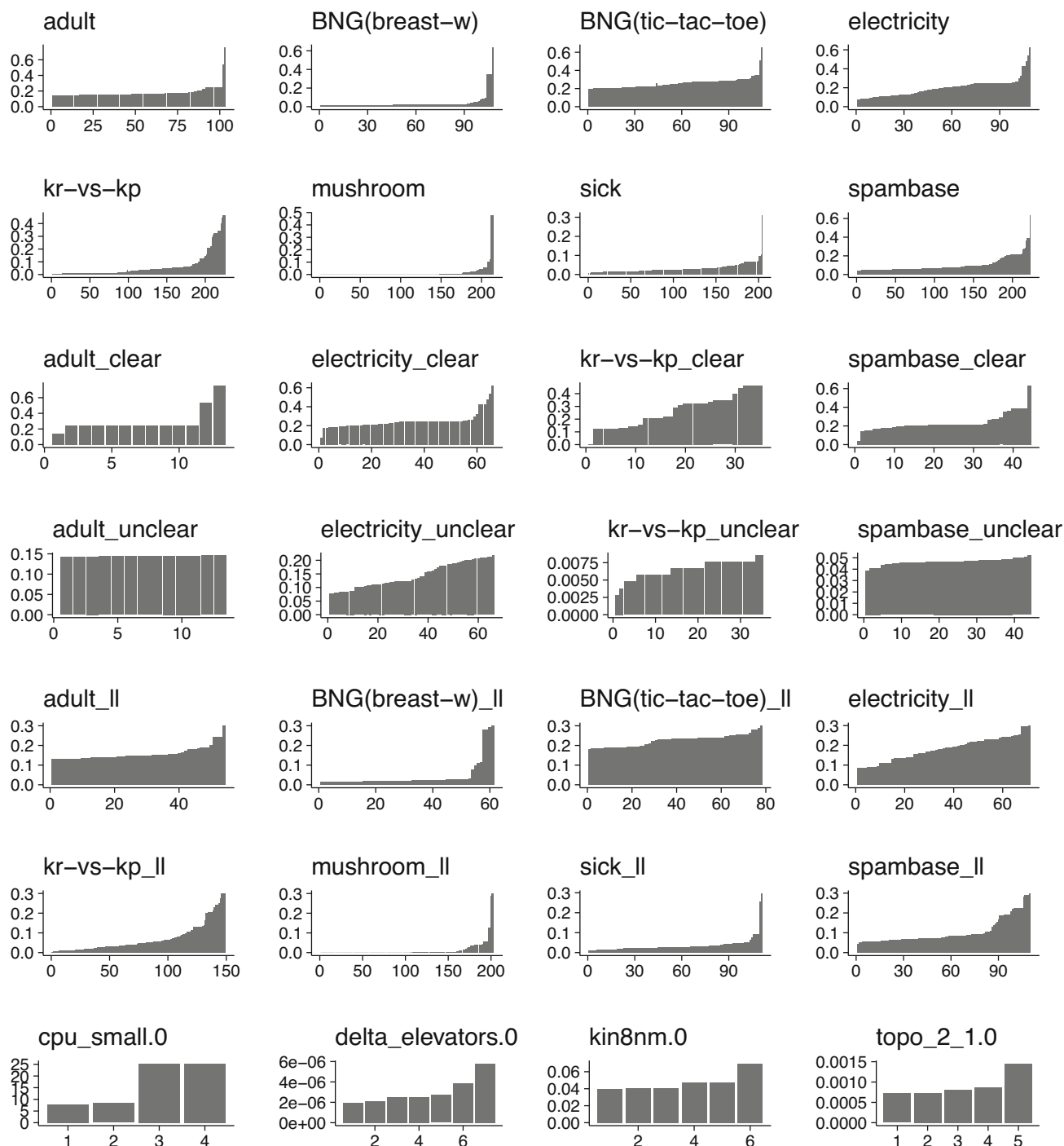


FIGURE A1 Empirical losses (y-axis) of models (x-axis) for datasets used in the experiments.

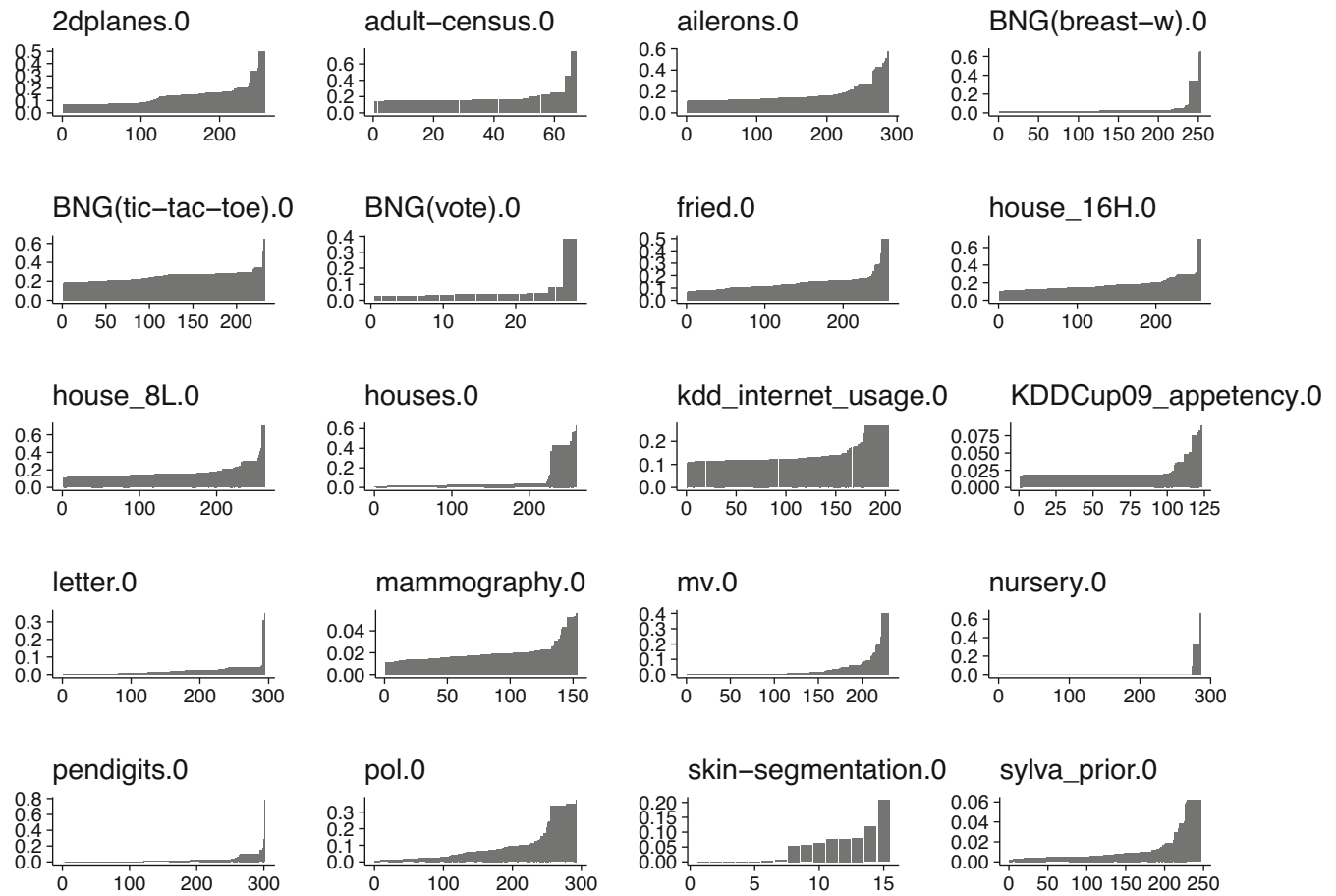


FIGURE A2 Empirical losses (y-axis) of models (x-axis) for datasets used in the experiments.

TABLE A1 OpenML IDs for datasets used in the experiments

Loss	Name	Data ID	Task ID
0-1, LL	adult	179	2117
0-1, LL	BNG (breast-w)	251	2328
0-1, LL	BNG (tic-tac-toe)	137	322
0-1, LL	electricity	151	336
0-1, LL	kr-vs-kp	3	233
0-1, LL	mushroom	24	254
0-1, LL	sick	38	3043
0-1, LL	spambase	44	273
MSE	cpu_small	227	2315
MSE	delta_elevators	198	2289
MSE	kin8nm	189	2280
MSE	topo_2_1	422	4796
0-1	2dplanes	727	3593
0-1	adult-census	1119	3953
0-1	aileron	734	3600
0-1	BNG (breast-w)	251	2142
0-1	BNG (tic-tac-toe)	137	206
0-1	BNG (vote)	143	212
0-1	fried	901	3764
0-1	house_16H	821	3686
0-1	house_8L	843	3708
0-1	houses	823	3688
0-1	kdd_internet_usage	981	3844
0-1	KDDCup09_appetency	1111	3945
0-1	letter	977	3840
0-1	mammography	310	3483
0-1	mv	881	3745
0-1	nursery	959	3822
0-1	pendigits	1019	3882
0-1	pol	722	3588
0-1	skin-segmentation	1502	9965
0-1	sylvia_prior	1040	3893

TABLE A2 Results of running BSV with $\alpha = 0.05$ and 0–1 loss. See the text for column descriptions.

Dataset	N	$k(k_\delta)$	δ	T	$E[n] \text{ (sd } (n))$	$E[n]/N$	error	power _{emp}
adult	16117	103 (34)	0.01	128	6977 (5085)	43%	0.7%	98.7%
adult	16117	103 (34)	0.01	512	5819 (4391)	36%	0.5%	99.0%
adult	16117	103 (91)	0.1	128	280 (74)	2%	0.1%	100.0%
adult	16117	103 (91)	0.1	512	1024 (0)	6%	0.1%	100.0%
BNG (breast-w)	12990	108 (77)	0.01	128	1254 (1150)	10%	0.7%	99.4%
BNG (breast-w)	12990	108 (77)	0.01	512	1677 (970)	13%	0.4%	99.7%
BNG (breast-w)	12990	108 (103)	0.1	128	257 (14)	2%	0.1%	100.0%
BNG (breast-w)	12990	108 (103)	0.1	512	1024 (0)	8%	0.1%	100.0%
BNG (tic-tac-toe)	12990	111 (19)	0.01	128	9484 (4343)	73%	0.5%	89.2%
BNG (tic-tac-toe)	12990	111 (19)	0.01	512	8068 (4400)	62%	0.4%	93.0%
BNG (tic-tac-toe)	12990	111 (95)	0.1	128	298 (103)	2%	0.1%	100.0%
BNG (tic-tac-toe)	12990	111 (95)	0.1	512	1024 (0)	8%	0.1%	100.0%
electricity	14952	109 (8)	0.01	128	2477 (2531)	17%	0.9%	99.2%
electricity	14952	109 (8)	0.01	512	2431 (1974)	16%	0.3%	99.8%
electricity	14952	109 (44)	0.1	128	271 (63)	2%	0.1%	100.0%
electricity	14952	109 (44)	0.1	512	1024 (0)	7%	0.1%	100.0%
kr-vs-kp	1054	226 (85)	0.01	128	525 (288)	50%	0.6%	98.9%
kr-vs-kp	1054	226 (85)	0.01	512	1033 (14)	98%	0.1%	97.8%
kr-vs-kp	1054	226 (192)	0.1	128	256 (8)	24%	0.1%	100.0%
kr-vs-kp	1054	226 (192)	0.1	512	1024 (0)	97%	0.1%	100.0%
mushroom	2680	215 (176)	0.01	128	262 (39)	10%	0.6%	99.5%
mushroom	2680	215 (176)	0.01	512	1025 (32)	38%	0.1%	100.0%
mushroom	2680	215 (207)	0.1	128	256 (0)	10%	0.1%	100.0%
mushroom	2680	215 (207)	0.1	512	1024 (0)	38%	0.1%	100.0%
sick	1244	204 (55)	0.01	128	758 (372)	61%	0.7%	97.2%
sick	1244	204 (55)	0.01	512	1105 (106)	89%	0.1%	95.5%
sick	1244	204 (203)	0.1	128	257 (11)	21%	0.1%	100.0%
sick	1244	204 (203)	0.1	512	1024 (0)	82%	0.1%	100.0%
spambase	1518	223 (39)	0.01	128	1138 (445)	75%	0.7%	79.8%
spambase	1518	223 (39)	0.01	512	1328 (240)	88%	0.1%	74.3%
spambase	1518	223 (180)	0.1	128	262 (40)	17%	0.1%	100.0%
spambase	1518	223 (180)	0.1	512	1024 (0)	67%	0.1%	100.0%

TABLE A3 Results of running BSV with $\alpha = 0.05$ and 0–1 loss. See the text for column descriptions.

Dataset	N	$k(k_\delta)$	δ	T	$E[n] \text{ (sd } (n))$	$E[n]/N$	error	power _{emp}
adult_clear	16117	13 (1)	0.01	128	301 (101)	2%	0.1%	100.0%
adult_clear	16117	13 (1)	0.01	512	1024 (0)	6%	0.1%	100.0%
adult_clear	16117	13 (1)	0.1	128	257 (14)	2%	0.1%	100.0%
adult_clear	16117	13 (1)	0.1	512	1024 (0)	6%	0.1%	100.0%
adult_unclear	16117	13 (13)	0.01	128	5502 (5082)	34%	0.1%	99.6%
adult_unclear	16117	13 (13)	0.01	512	4953 (4416)	31%	0.1%	99.7%
adult_unclear	16117	13 (13)	0.1	128	258 (21)	2%	0.1%	100.0%
adult_unclear	16117	13 (13)	0.1	512	1024 (0)	6%	0.1%	100.0%
electricity_clear	14952	66 (1)	0.01	128	272 (66)	2%	0.1%	100.0%
electricity_clear	14952	66 (1)	0.01	512	1024 (0)	7%	0.1%	100.0%
electricity_clear	14952	66 (1)	0.1	128	257 (18)	2%	0.1%	100.0%
electricity_clear	14952	66 (1)	0.1	512	1024 (0)	7%	0.1%	100.0%
electricity_unclear	14952	66 (8)	0.01	128	2477 (2531)	17%	0.9%	99.2%
electricity_unclear	14952	66 (8)	0.01	512	2431 (1974)	16%	0.3%	99.8%
electricity_unclear	14952	66 (44)	0.1	128	271 (63)	2%	0.1%	100.0%
electricity_unclear	14952	66 (44)	0.1	512	1024 (0)	7%	0.1%	100.0%
kr-vs-kp_clear	1054	35 (1)	0.01	128	256 (0)	24%	0.1%	100.0%
kr-vs-kp_clear	1054	35 (1)	0.01	512	1024 (0)	97%	0.1%	100.0%
kr-vs-kp_clear	1054	35 (1)	0.1	128	256 (0)	24%	0.1%	100.0%
kr-vs-kp_clear	1054	35 (1)	0.1	512	1024 (0)	97%	0.1%	100.0%
kr-vs-kp_unclear	1054	35 (35)	0.01	128	459 (270)	44%	0.1%	99.9%
kr-vs-kp_unclear	1054	35 (35)	0.01	512	1033 (14)	98%	0.1%	96.9%
kr-vs-kp_unclear	1054	35 (35)	0.1	128	256 (0)	24%	0.1%	100.0%
kr-vs-kp_unclear	1054	35 (35)	0.1	512	1024 (0)	97%	0.1%	100.0%
spambase_clear	1518	44 (1)	0.01	128	257 (16)	17%	0.1%	100.0%
spambase_clear	1518	44 (1)	0.01	512	1024 (0)	67%	0.1%	100.0%
spambase_clear	1518	44 (1)	0.1	128	256 (0)	17%	0.1%	100.0%
spambase_clear	1518	44 (1)	0.1	512	1024 (0)	67%	0.1%	100.0%
spambase_unclear	1518	44 (39)	0.01	128	1133 (449)	75%	0.3%	77.6%
spambase_unclear	1518	44 (39)	0.01	512	1327 (241)	87%	0.1%	77.1%
spambase_unclear	1518	44 (44)	0.1	128	256 (8)	17%	0.1%	100.0%
spambase_unclear	1518	44 (44)	0.1	512	1024 (0)	67%	0.1%	100.0%

TABLE A4 Results of running BSV with $\alpha = 0.05$ and 0–1 loss. See the text for column descriptions.

Dataset	<i>N</i>	<i>k</i> (<i>k</i> _{δ})	δ	<i>T</i>	E[<i>n</i>] (sd (<i>n</i>))	E[<i>n</i>]/ <i>N</i>	error	power _{emp}
2dplanes.0	4077	257 (88)	0.01	128	3059 (1405)	75%	0.4%	81.7%
2dplanes.0	4077	257 (88)	0.01	256	3043 (1382)	75%	0.1%	81.2%
2dplanes.0	4077	257 (121)	0.05	128	372 (185)	9%	0.1%	100.0%
2dplanes.0	4077	257 (121)	0.05	256	556 (143)	14%	0.1%	100.0%
adult-census.0	3257	67 (6)	0.01	128	1551 (982)	48%	4.6%	94.8%
adult-census.0	3257	67 (6)	0.01	256	1699 (940)	52%	2.5%	96.8%
adult-census.0	3257	67 (51)	0.05	128	460 (286)	14%	0.1%	100.0%
adult-census.0	3257	67 (51)	0.05	256	658 (281)	20%	0.1%	100.0%
aileron.0	1375	288 (17)	0.01	128	1274 (243)	93%	1.8%	54.7%
aileron.0	1375	288 (17)	0.01	256	1288 (206)	94%	0.8%	55.2%
aileron.0	1375	288 (187)	0.05	128	519 (287)	38%	0.1%	100.0%
aileron.0	1375	288 (187)	0.05	256	655 (236)	48%	0.1%	100.0%
BNG (breast-w).0	3937	253 (182)	0.01	128	1411 (1140)	36%	1.0%	98.7%
BNG (breast-w).0	3937	253 (182)	0.01	256	1506 (1035)	38%	0.4%	99.6%
BNG (breast-w).0	3937	253 (233)	0.05	128	285 (86)	7%	0.1%	100.0%
BNG (breast-w).0	3937	253 (233)	0.05	256	515 (40)	13%	0.1%	100.0%
BNG (tic-tac-toe).0	3937	232 (15)	0.01	128	3578 (962)	91%	3.0%	62.6%
BNG (tic-tac-toe).0	3937	232 (15)	0.01	256	3500 (983)	89%	1.9%	63.4%
BNG (tic-tac-toe).0	3937	232 (95)	0.05	128	649 (457)	16%	0.2%	99.9%
BNG (tic-tac-toe).0	3937	232 (95)	0.05	256	757 (345)	19%	0.1%	100.0%
BNG (vote).0	13108	28 (10)	0.01	128	793 (767)	6%	2.8%	97.3%
BNG (vote).0	13108	28 (10)	0.01	256	949 (701)	7%	1.4%	98.7%
BNG (vote).0	13108	28 (24)	0.05	128	279 (81)	2%	0.1%	100.0%
BNG (vote).0	13108	28 (24)	0.05	256	522 (72)	4%	0.1%	100.0%
fried.0	4077	258 (15)	0.01	128	3060 (1377)	75%	0.9%	80.3%
fried.0	4077	258 (15)	0.01	256	2971 (1368)	73%	1.3%	80.2%
fried.0	4077	258 (111)	0.05	128	428 (252)	10%	0.1%	100.0%
fried.0	4077	258 (111)	0.05	256	601 (214)	15%	0.1%	100.0%
house_16H.0	2279	257 (22)	0.01	128	2101 (499)	92%	0.6%	20.2%
house_16H.0	2279	257 (22)	0.01	256	2060 (501)	90%	0.1%	27.1%
house_16H.0	2279	257 (126)	0.05	128	498 (304)	22%	0.1%	100.0%
house_16H.0	2279	257 (126)	0.05	256	652 (257)	29%	0.1%	100.0%
house_8L.0	2279	262 (28)	0.01	128	1948 (605)	85%	1.6%	50.8%
house_8L.0	2279	262 (28)	0.01	256	2016 (511)	88%	0.4%	50.0%
house_8L.0	2279	262 (173)	0.05	128	493 (306)	22%	0.1%	100.0%
house_8L.0	2279	262 (173)	0.05	256	644 (247)	28%	0.1%	100.0%
houses.0	2064	261 (76)	0.01	128	1036 (680)	50%	1.1%	98.6%
houses.0	2064	261 (76)	0.01	256	1112 (601)	54%	0.4%	99.1%
houses.0	2064	261 (223)	0.05	128	285 (84)	14%	0.1%	100.0%
houses.0	2064	261 (223)	0.05	256	519 (58)	25%	0.1%	100.0%

TABLE A5 Results of running BSV with $\alpha = 0.05$ and 0–1 loss. See the text for column descriptions.

Dataset	N	$k(k_\delta)$	δ	T	$E[n]$ (sd (n))	$E[n]/N$	error	power _{emp}
kdd_internet_usage.0	1011	203 (54)	0.01	128	977 (142)	97%	3.7%	14.0%
kdd_internet_usage.0	1011	203 (54)	0.01	256	997 (82)	99%	5.2%	9.8%
kdd_internet_usage.0	1011	203 (161)	0.05	128	482 (258)	48%	0.2%	99.6%
kdd_internet_usage.0	1011	203 (161)	0.05	256	619 (205)	61%	0.1%	99.4%
KDDCup09_appetency.0	5000	123 (104)	0.01	128	401 (268)	8%	0.1%	100.0%
KDDCup09_appetency.0	5000	123 (104)	0.01	256	614 (311)	12%	0.1%	100.0%
KDDCup09_appetency.0	5000	123 (116)	0.05	128	260 (31)	5%	0.1%	100.0%
KDDCup09_appetency.0	5000	123 (116)	0.05	256	513 (16)	10%	0.1%	100.0%
letter.0	2000	294 (123)	0.01	128	406 (246)	20%	0.4%	99.7%
letter.0	2000	294 (123)	0.01	256	656 (304)	33%	0.1%	100.0%
letter.0	2000	294 (290)	0.05	128	263 (42)	13%	0.1%	100.0%
letter.0	2000	294 (290)	0.05	256	513 (16)	26%	0.1%	100.0%
mammography.0	1119	153 (119)	0.01	128	583 (327)	52%	0.4%	95.1%
mammography.0	1119	153 (119)	0.01	256	766 (272)	68%	0.1%	93.9%
mammography.0	1119	153 (153)	0.05	128	266 (49)	24%	0.1%	100.0%
mammography.0	1119	153 (153)	0.05	256	513 (23)	46%	0.1%	100.0%
mv.0	4077	230 (137)	0.01	128	352 (182)	9%	0.2%	99.9%
mv.0	4077	230 (137)	0.01	256	603 (249)	15%	0.4%	99.7%
mv.0	4077	230 (181)	0.05	128	258 (21)	6%	0.1%	100.0%
mv.0	4077	230 (181)	0.05	256	512 (0)	13%	0.1%	100.0%
nursery.0	1296	286 (273)	0.01	128	256 (0)	20%	0.1%	100.0%
nursery.0	1296	286 (273)	0.01	256	512 (0)	40%	0.1%	100.0%
nursery.0	1296	286 (273)	0.05	128	256 (0)	20%	0.1%	100.0%
nursery.0	1296	286 (273)	0.05	256	512 (0)	40%	0.1%	100.0%
pendigits.0	1100	301 (147)	0.01	128	382 (200)	35%	1.6%	98.5%
pendigits.0	1100	301 (147)	0.01	256	598 (192)	54%	0.1%	100.0%
pendigits.0	1100	301 (257)	0.05	128	259 (28)	24%	0.1%	100.0%
pendigits.0	1100	301 (257)	0.05	256	513 (16)	47%	0.1%	100.0%
pol.0	1500	293 (42)	0.01	128	848 (460)	57%	1.4%	93.2%
pol.0	1500	293 (42)	0.01	256	990 (400)	66%	0.3%	93.0%
pol.0	1500	293 (132)	0.05	128	281 (76)	19%	0.1%	100.0%
pol.0	1500	293 (132)	0.05	256	520 (64)	35%	0.1%	100.0%
skin-segmentation.0	24506	15 (7)	0.01	128	291 (97)	1%	0.1%	100.0%
skin-segmentation.0	24506	15 (7)	0.01	256	531 (98)	2%	0.1%	100.0%
skin-segmentation.0	24506	15 (7)	0.05	128	257 (11)	1%	0.1%	100.0%
skin-segmentation.0	24506	15 (7)	0.05	256	512 (0)	2%	0.1%	100.0%
sylva_prior.0	1440	247 (189)	0.01	128	433 (260)	30%	0.2%	99.9%
sylva_prior.0	1440	247 (189)	0.01	256	671 (270)	47%	0.1%	100.0%
sylva_prior.0	1440	247 (226)	0.05	128	258 (24)	18%	0.1%	100.0%
sylva_prior.0	1440	247 (226)	0.05	256	512 (0)	36%	0.1%	100.0%

TABLE A6 Results of running BSV with $\alpha = 0.05$ and log loss. See the text for column descriptions.

Dataset	N	$k(k_\delta)$	δ	T	$E[n](sd(n))$	$E[n]/N$	error	power _{emp}
adult_ll	16117	54 (14)	0.01	128	688 (556)	4%	1.2%	98.9%
adult_ll	16117	54 (14)	0.01	256	835 (590)	5%	0.7%	99.4%
adult_ll	16117	54 (50)	0.1	128	259 (29)	2%	0.1%	100.0%
adult_ll	16117	54 (50)	0.1	256	512 (0)	3%	0.1%	100.0%
BNG (breast-w)_ll	12990	61 (37)	0.01	128	522 (376)	4%	0.5%	99.6%
BNG (breast-w)_ll	12990	61 (37)	0.01	256	702 (348)	5%	0.2%	99.9%
BNG (breast-w)_ll	12990	61 (56)	0.1	128	256 (8)	2%	0.1%	100.0%
BNG (breast-w)_ll	12990	61 (56)	0.1	256	512 (0)	4%	0.1%	100.0%
BNG (tic-tac-toe)_ll	12990	78 (17)	0.01	128	1938 (1879)	15%	0.9%	99.2%
BNG (tic-tac-toe)_ll	12990	78 (17)	0.01	256	1961 (1753)	15%	0.3%	99.8%
BNG (tic-tac-toe)_ll	12990	78 (76)	0.1	128	259 (25)	2%	0.1%	100.0%
BNG (tic-tac-toe)_ll	12990	78 (76)	0.1	256	512 (0)	4%	0.1%	100.0%
electricity_ll	14952	71 (8)	0.01	128	340 (165)	2%	0.4%	99.7%
electricity_ll	14952	71 (8)	0.01	256	550 (146)	4%	0.2%	99.9%
electricity_ll	14952	71 (36)	0.1	128	256 (0)	2%	0.1%	100.0%
electricity_ll	14952	71 (36)	0.1	256	512 (0)	3%	0.1%	100.0%
kr-vs-kp_ll	1054	149 (26)	0.01	128	404 (225)	38%	0.1%	100.0%
kr-vs-kp_ll	1054	149 (26)	0.01	256	647 (226)	61%	0.1%	99.9%
kr-vs-kp_ll	1054	149 (116)	0.1	128	256 (0)	24%	0.1%	100.0%
kr-vs-kp_ll	1054	149 (116)	0.1	256	512 (0)	49%	0.1%	100.0%
mushroom_ll	2680	203 (165)	0.01	128	256 (0)	10%	0.1%	100.0%
mushroom_ll	2680	203 (165)	0.01	256	512 (0)	19%	0.1%	100.0%
mushroom_ll	2680	203 (196)	0.1	128	256 (0)	10%	0.1%	100.0%
mushroom_ll	2680	203 (196)	0.1	256	512 (0)	19%	0.1%	100.0%
sick_ll	1244	111 (22)	0.01	128	564 (344)	45%	0.2%	98.8%
sick_ll	1244	111 (22)	0.01	256	761 (292)	61%	0.1%	98.4%
sick_ll	1244	111 (109)	0.1	128	256 (0)	21%	0.1%	100.0%
sick_ll	1244	111 (109)	0.1	256	512 (0)	41%	0.1%	100.0%
spambase_ll	1518	110 (7)	0.01	128	671 (379)	44%	2.8%	97.3%
spambase_ll	1518	110 (7)	0.01	256	778 (318)	51%	1.1%	99.0%
spambase_ll	1518	110 (87)	0.1	128	258 (23)	17%	0.1%	100.0%
spambase_ll	1518	110 (87)	0.1	256	512 (0)	34%	0.1%	100.0%

TABLE A7 Results of running BSV with $\alpha = 0.05$ and MSE loss. See the text for column descriptions.

Dataset	N	$k(k_\delta)$	δ	T	$E[n] (sd(n))$	$E[n]/N$	error	power _{emp}
cpu_small.0	820	4 (1)	0.1	64	463 (266)	56%	0.1%	99.7%
cpu_small.0	820	4 (1)	0.1	128	478 (217)	58%	0.1%	99.9%
cpu_small.0	820	4 (2)	5	64	128 (0)	16%	0.1%	100.0%
cpu_small.0	820	4 (2)	5	128	256 (0)	31%	0.1%	100.0%
delta_elevators.0	952	7 (5)	1e-06	64	137 (33)	14%	0.1%	100.0%
delta_elevators.0	952	7 (5)	1e-06	128	259 (27)	27%	0.1%	100.0%
delta_elevators.0	952	7 (2)	2e-07	64	401 (268)	42%	0.3%	99.6%
delta_elevators.0	952	7 (2)	2e-07	128	441 (239)	46%	0.1%	99.8%
kin8nm.0	820	6 (3)	0.001	64	483 (299)	59%	0.6%	92.0%
kin8nm.0	820	6 (3)	0.001	128	571 (248)	70%	0.1%	92.4%
kin8nm.0	820	6 (5)	0.01	64	136 (34)	17%	0.1%	100.0%
kin8nm.0	820	6 (5)	0.01	128	259 (28)	32%	0.1%	100.0%
topo_2_1.0	889	5 (4)	0.0002	64	195 (120)	22%	0.1%	100.0%
topo_2_1.0	889	5 (4)	0.0002	128	313 (122)	35%	0.1%	100.0%
topo_2_1.0	889	5 (2)	8e-05	64	522 (325)	59%	0.3%	76.6%
topo_2_1.0	889	5 (2)	8e-05	128	586 (285)	66%	0.4%	77.4%